SAPHO

Manual do Usuário

Concepção, programação, simulação e visualização
de *soft-processors* com a IDE AURORA

NIPS-CERN

Núcleo de Instrumentação e Processamento de Sinais
Universidade Federal de Juiz de Fora

`https://nipscern.com/sapho`

Documento vivo. Acompanha as versões da plataforma.

Sumário

Prefácio **ix**

I Primeiros passos **1**

1 O que é o SAPHO **3**
1.1 O que é um soft-processor . . . 3
1.2 As peças da plataforma . . . 3
1.3 O fluxo de trabalho de ponta a ponta . . . 4
1.4 O exemplo que acompanha este manual . . . 4
1.5 O laboratório por trás: NIPS-CERN . . . 4

2 Instalação **7**
2.1 Baixando e instalando . . . 7
2.2 O que é instalado, e onde . . . 8
2.3 Primeira execução . . . 8
2.4 Desinstalação . . . 8

3 Tour pela interface **9**
3.1 A barra de título e a toolbar . . . 9
3.2 O painel da árvore de arquivos . . . 10
3.3 O editor . . . 10
3.4 Os terminais . . . 11
3.5 A barra de status . . . 11
3.6 Paleta de comandos, notificações e tooltips . . . 11
3.7 A tela de boas-vindas . . . 11

II Projetos e processadores **13**

4 Projetos **15**
4.1 Criando um projeto . . . 15
4.2 O arquivo `.spf` . . . 15
4.3 Abrindo um projeto . . . 16
4.4 Arquivos do projeto: quem é quem . . . 16

4.5 As três visões da árvore . . . 17
4.6 Manutenção do projeto . . . 18

5 O processador SAPHO 19
5.1 Visão geral da arquitetura . . . 19
5.2 Os parâmetros fundamentais . . . 19
5.3 O ponto flutuante do SAPHO . . . 20
5.4 Como o programa vira circuito . . . 21
5.5 Entrada, saída e o mundo externo . . . 21
5.6 Um processador por tarefa . . . 21

6 Criando processadores: o Hub de Processadores 23
6.1 Os campos do formulário . . . 23
6.2 O que é gerado . . . 24
6.3 Configurações de simulação do processador . . . 25
6.4 Renomear e excluir processadores . . . 25

III A linguagem C± 27

7 C±: fundamentos 29
7.1 A estrutura de um programa . . . 29
7.2 Diretivas: configurando o processador no próprio código . . . 30
7.3 Constantes com `#define` . . . 31
7.4 Comentários . . . 31
7.5 Onde o arquivo vive . . . 31

8 C±: tipos, operadores e controle 33
8.1 Tipos de dados . . . 33
8.1.1 Literais complexos e o `i` reservado . . . 33
8.1.2 Conversões entre tipos . . . 33
8.2 Operadores . . . 34
8.2.1 Operadores custam hardware . . . 34
8.3 Estruturas de controle . . . 34
8.4 Funções . . . 35
8.5 Arrays . . . 36
8.6 Variáveis e escopo . . . 36

9 C±: entrada, saída e biblioteca padrão 37
9.1 Entrada e saída . . . 37
9.2 Funções especiais . . . 37
9.3 Funções não lineares e de arredondamento . . . 37

9.4 Funções para complexos . . . 38
9.5 Álgebra linear em notação de Dirac . . . 38

10 C±: recursos avançados **39**
10.1 Interrupção: `#PRACA` . . . 39
10.2 Sincronização com o hardware: `#TOAQUI` . . . 39
10.3 FFT e o endereçamento bit-reverso . . . 39
10.4 Quanto custa cada construção . . . 40
10.5 Links entre processadores . . . 40
10.6 O caminho C++ . . . 41
10.7 As mensagens do compilador . . . 41

IV O editor **43**

11 O editor **45**
11.1 Abas . . . 45
11.2 Editor dividido . . . 45
11.3 Realce de sintaxe . . . 46
11.4 Diagnósticos e navegação . . . 46
11.5 Formatação automática . . . 46
11.6 Busca no projeto . . . 46
11.7 Seleção e IA . . . 46
11.8 Conflitos com o disco . . . 47
11.9 Atalhos de teclado . . . 47

V Compilação e simulação **49**

12 Compilação **51**
12.1 O pipeline do YANC em duas etapas . . . 51
12.2 Os botões de compilação . . . 51
12.3 Onde cada artefato aparece . . . 52

13 Os terminais **55**
13.1 Quem escreve onde . . . 55
13.2 Cartões, filtros e o modo verboso . . . 55
13.3 TCMD: o shell integrado . . . 56

14 Simulação **57**
14.1 Dois motores: Icarus Verilog e Verilator . . . 57
14.2 Dois tipos de testbench: Verilog e cocotb . . . 57

14.3 Parâmetros da simulação . . . 58
14.4 Escolhendo os sinais das ondas . . . 58
14.5 O seletor de `.gtkw` . . . 59
14.6 Rodando . . . 59
14.7 Teste de hardware . . . 59
14.8 Simulando o `media_movel` . . . 59

VI Visualização **61**

15 Formas de onda: GTKWave e Surfer **63**
15.1 O que a AURORA prepara para você . . . 63
15.2 Escolhendo o visualizador . . . 64
15.3 GTKWave . . . 64
15.4 Surfer . . . 64

16 PRISM: o visualizador de RTL **67**
16.1 Como abrir . . . 67
16.2 Navegando no diagrama . . . 67
16.3 Modo simulação . . . 68

VII Aurora Intelligence **69**

17 Aurora Intelligence **71**
17.1 Abrindo o painel . . . 71
17.2 Configurando um provedor . . . 71
17.3 O composer . . . 72
17.4 Ferramentas e permissões . . . 72
17.5 Conversas . . . 73
17.6 Para que ela é boa no fluxo SAPHO . . . 73

VIII Ferramentas de apoio **75**

18 Dagr: controle de versão **77**
18.1 Abrindo . . . 77
18.2 A aba Changes . . . 77
18.3 A aba History . . . 78
18.4 Ramos e stashes . . . 78
18.5 GitHub . . . 78

19 Configurações **79**

IX Manutenção da plataforma **81**

20 Atualizações automáticas **83**
20.1 Como funciona, do seu ponto de vista 83
20.2 A toolchain acompanha 84
20.3 Este manual também se atualiza 84

21 Solução de problemas **85**
21.1 Compilação e simulação 85
21.2 Projeto e arquivos 86
21.3 Visualizadores 87
21.4 Aurora Intelligence 87
21.5 Onde estão os registros 87

X Apêndices **89**

A Referência de diretivas **91**
A.1 Diretivas de configuração 91
A.2 Diretivas comportamentais 92
A.3 Pré-processador 92

B Referência da biblioteca padrão **93**
B.1 Entrada e saída 93
B.2 Links entre processadores 93
B.3 Funções especiais 94
B.4 Não lineares e arredondamento 94
B.5 Complexos 95
B.6 Notação de Dirac 95
B.7 Famílias de instruções do processador 95

C Atalhos de teclado **97**

D Glossário **99**

Prefácio

Este livro é o manual de uso da plataforma SAPHO, da instalação ao primeiro processador funcionando em simulação, e dali aos recursos mais avançados da IDE AURORA e da linguagem C±. Ele foi escrito para quem nunca viu o SAPHO. Nenhum conhecimento prévio da plataforma é assumido, embora alguma familiaridade com a linguagem C e com a ideia geral de circuitos digitais torne a leitura mais confortável.

O texto nasceu de um princípio simples: toda afirmação sobre o comportamento da plataforma foi extraída do código-fonte dos repositórios oficiais na data da edição, e não de documentos intermediários. Quando o manual diz que um botão existe, que um campo aceita determinado valor ou que uma construção da linguagem não compila, é porque o código diz exatamente isso.

Como este livro está organizado

Cada componente do SAPHO é, na prática, um projeto por si só, e por isso o livro se divide em partes que funcionam como pequenas documentações independentes. A Parte I apresenta a plataforma, a instalação e um passeio pela interface. A Parte II trata dos projetos e do processador: o arquivo `.spf`, a arquitetura da máquina e o formulário que a cria. A Parte III é a referência completa da linguagem C±, da estrutura de um programa aos recursos avançados. A Parte IV cobre o editor e todo o apoio à edição de código. A Parte V percorre a compilação, os terminais e a simulação; a Parte VI, os dois visualizadores de formas de onda e o visualizador de RTL. A Parte VII é dedicada à assistente de IA integrada, e a Parte VIII às ferramentas de apoio, o controle de versão e as configurações. A Parte IX fecha o volume com o atualizador automático e um capítulo de solução de problemas. Os apêndices reúnem as referências rápidas: diretivas, biblioteca padrão, atalhos de teclado e glossário.

Um exemplo único costura os capítulos: o processador `media_movel`, um filtro de média móvel construído do zero no projeto `MeuFiltro`. Ele nasce no Capítulo 6, ganha código na Parte III, compila na Parte V e aparece nas formas de onda e no diagrama de RTL na Parte VI.

Convenções usadas neste livro

Os elementos clicáveis da interface aparecem em fonte sem serifa, como em Novo Projeto; os atalhos de teclado, em moldura, como Ctrl+S; nomes de comandos, diretivas e trechos de código, em fonte monoespaçada, como `main()`; nomes de arquivos e caminhos, como `media_movel.cmm`; e os componentes da plataforma, em versalete, como YANC. Termos estrangeiros consagrados (*testbench*, *soft-processor*) permanecem em itálico.

Quatro tipos de caixa pontuam o texto, sempre à margem do assunto principal:

> NOTA
> Dicas e observações que enriquecem o assunto ao lado.

> INFORMAÇÃO
> Contexto adicional, referências cruzadas e detalhes de bastidores.

> AVISO
> Restrições e comportamentos que costumam surpreender.

> CUIDADO
> Ações destrutivas ou erros de consequências sérias.

Um documento vivo

Este manual acompanha o desenvolvimento da plataforma e é atualizado junto com a AURORA, o YANC e os demais componentes. Se você encontrar divergência entre o texto e o software, o software mudou primeiro: procure a edição mais recente do manual ou registre o problema nos repositórios da organização `nipscernlab`, no GitHub.

PARTE I

Primeiros passos

CAPÍTULO 1

O que é o SAPHO

O SAPHO (*Scalable-Architecture Processor for Hardware Optimization*) é uma plataforma completa para criar, programar, simular e visualizar *soft-processors*. O usuário escreve um programa numa linguagem parecida com C, chamada C± (lê-se "C mais-menos"), e a plataforma gera um processador dedicado em Verilog, sob medida para aquele programa, pronto para ser sintetizado em FPGA (*Field-Programmable Gate Array*). Tudo acontece dentro de uma única aplicação de desktop, a IDE AURORA (*Advanced Utility Running Optimized Resource Architectures*), que roda localmente no Windows, sem depender de nuvem.

1.1 O que é um soft-processor

Um processador convencional, como o do seu computador, é um circuito fixo: sua arquitetura foi decidida na fábrica e não muda. Um *soft-processor*, por outro lado, é um processador descrito em uma linguagem de descrição de hardware (no caso, o Verilog) e implementado na malha reconfigurável de um FPGA. A arquitetura, isto é, a largura da palavra, o conjunto de instruções, a quantidade de portas e o tamanho das memórias, passa a ser uma escolha do projetista, que pode alterá-la e recompilá-la em minutos.

O SAPHO leva essa ideia ao extremo: o processador gerado contém apenas o hardware que o programa usa. Se o programa não divide, o divisor não existe no circuito final; se não usa ponto flutuante, nenhuma lógica de ponto flutuante é gerada. O resultado é um processador enxuto, previsível e eficiente, feito para a instrumentação científica e para o processamento de sinais em tempo real.

1.2 As peças da plataforma

O nome SAPHO designa tanto o processador quanto o guarda-chuva que reúne os componentes da plataforma, e cada componente tem o seu módulo próprio neste manual. A AURORA é a IDE de desktop, o programa que você instala e abre: nela vivem o editor, o gerenciador de projetos, os botões de compilação e simulação, os terminais e os visualizadores. O YANC (*Yet Another Compiler*) é a

suíte de compiladores que trabalha por baixo dos panos, traduzindo o programa C± (ou C++) no processador em Verilog, nas imagens de memória e no *testbench*. O processador SAPHO propriamente dito é o circuito parametrizável que o YANC emite, descrito no Capítulo 5.

Em volta desse núcleo orbitam as ferramentas de apoio. Um pacote portátil de ferramentas abertas de EDA (*Electronic Design Automation*) acompanha a instalação: o simulador Icarus Verilog, o Verilator, o Yosys, o GCC e o Python com cocotb. O PRISM (*Processor Rendering Interface for Schematic Models*) é o visualizador de RTL embutido, que desenha o circuito sintetizado como um diagrama navegável (Capítulo 16). As formas de onda das simulações podem ser inspecionadas em dois visualizadores, o GTKWave e o Surfer (Capítulo 15). Por fim, a Aurora Intelligence é a assistente de IA integrada, capaz de conversar sobre o projeto e de agir sobre a IDE por ferramentas curadas (Capítulo 17).

1.3 O fluxo de trabalho de ponta a ponta

O caminho de quem usa o SAPHO é sempre o mesmo, e este manual o percorre na ordem natural. Cria-se um projeto, que é uma pasta governada por um arquivo `.spf` (Capítulo 4); dentro dele, cria-se um processador, escolhendo nome e parâmetros de arquitetura (Capítulo 6); programa-se o comportamento em C± no editor (Parte III); compila-se, e o YANC transforma o programa em Verilog, memórias e *testbench* (Capítulo 12); simula-se com o Icarus Verilog ou com o Verilator (Capítulo 14); e inspeciona-se o resultado, as formas de onda no GTKWave ou no Surfer e o circuito no PRISM (Parte VI). A síntese física no FPGA, com a ferramenta do fabricante (Quartus, Vivado), é a única etapa que acontece fora da AURORA: basta levar o `.v` e os `.mif` gerados.

1.4 O exemplo que acompanha este manual

Para que cada conceito apareça em contexto, o manual constrói um processador de exemplo do zero e o acompanha até a forma de onda final: o `media_movel`, um filtro de média móvel que lê amostras de um sinal por uma porta de entrada, calcula a média das últimas amostras e escreve o resultado numa porta de saída. É um exemplo pequeno, mas exercita o essencial: entrada e saída, aritmética, laço principal e simulação com estímulo externo.

1.5 O laboratório por trás: NIPS-CERN

O SAPHO é desenvolvido e mantido pelo NIPS-CERN, o Núcleo de Instrumentação e Processamento de Sinais da Universidade Federal de Juiz de Fora (UFJF), em Minas

Gerais. O laboratório atua em parceria com o CERN no experimento ATLAS do LHC (*Large Hadron Collider*), onde técnicas de processamento de sinais em FPGA, o habitat natural do SAPHO, são aplicadas à instrumentação dos calorímetros. A plataforma também sustenta o ensino na disciplina de Dispositivos Lógicos Programáveis (DLP) da UFJF, na qual os alunos projetam, simulam e inspecionam processadores de ponta a ponta sem depender de licenças proprietárias.

Todo o ecossistema é de código aberto, hospedado na organização `nipscernlab` no GitHub. O canal oficial de download para o usuário final é `https://nipscern.com/sapho`.

> NOTA
>
> Este é um documento vivo: ele acompanha as versões da plataforma e é atualizado junto com a AURORA, o YANC e os demais componentes. Se algo na sua tela não corresponde ao manual, verifique se a instalação está atualizada (Capítulo 20).

CAPÍTULO 2

Instalação

A AURORA é, hoje, um aplicativo exclusivamente Windows, para sistemas de 64 bits (Windows 10 ou 11). Reserve cerca de 2 GB de espaço em disco: o instalador já traz embutida toda a cadeia de ferramentas de compilação e simulação, de modo que nenhum outro programa precisa ser instalado.

2.1 Baixando e instalando

Acesse `https://nipscern.com/sapho` e baixe o instalador, um arquivo chamado `sapho-aurora-Setup-vX.Y.Z.exe`, onde `X.Y.Z` é a versão. Dê dois cliques no executável. O instalador é do tipo *one-click*: não há telas de opções nem escolha de pasta; ele instala, cria os atalhos na área de trabalho e no menu Iniciar e abre o aplicativo.

Durante a instalação, dois vínculos são registrados no sistema. A extensão `.spf`, dos arquivos de projeto do SAPHO, passa a abrir com a AURORA e ganha ícone próprio, de modo que dois cliques num projeto no Explorer o abrem diretamente. Também é registrado o protocolo `sapho://`, reservado a integrações via navegador.

FIGURA 1 *Página de download em nipscern.com/sapho*

Mostrar o botão de download do instalador.

> **NOTA**
>
> O instalador ainda não é assinado digitalmente, e o Windows SmartScreen pode exibir um aviso de editor desconhecido na primeira execução. Nesse caso, clique em Mais informações e depois em Executar assim mesmo.

2.2 O que é instalado, e onde

A pasta do aplicativo recebe o executável da AURORA e a pasta `components`, que reúne a cadeia de ferramentas completa: os compiladores do YANC, os simuladores Icarus Verilog e Verilator, o Yosys, o GTKWave, o Python com cocotb e os moldes de HDL do processador. Os dados do usuário ficam em `\%APPDATA\%\textbackslashAurora-IDE`: os registros de execução (`logs/main.log`), a lista de projetos recentes e o registro de versão usado na confirmação pós-atualização. As credenciais sensíveis, como as chaves de API da Aurora Intelligence e o token do GitHub, não vão para arquivo algum: são cifradas pelo cofre de credenciais do próprio Windows (DPAPI).

Os seus projetos, por sua vez, ficam onde você quiser; a AURORA pergunta a pasta ao criar cada um deles (Capítulo 4).

2.3 Primeira execução

Ao abrir, a AURORA exibe uma tela de *splash* com o progresso real de inicialização e, em seguida, a tela de boas-vindas: o fundo animado de aurora boreal, os botões Novo Projeto e Abrir Projeto e a lista de projetos recentes, vazia na primeira vez. A partir daí, o Capítulo 3 apresenta a interface, e o Capítulo 4 cria o primeiro projeto.

Cerca de seis segundos após abrir, a AURORA verifica silenciosamente se há uma versão mais nova; se houver, a janela de atualização aparece (Capítulo 20). Manter o SAPHO em dia não exige nada além de aceitar as atualizações quando oferecidas.

FIGURA 2 *Primeira abertura da AURORA: tela de splash e tela de boas-vindas*

Capturar a splash com a barra de progresso e, em seguida, a tela de boas-vindas com o fundo animado de aurora boreal.

2.4 Desinstalação

Use as Configurações do Windows, em Aplicativos, e desinstale o item Aurora-IDE. Os projetos criados por você não são apagados: eles pertencem às pastas que você escolheu.

CAPÍTULO 3

Tour pela interface

A janela da AURORA não tem a moldura padrão do Windows nem uma barra de menus tradicional. Toda a navegação vive numa barra de título personalizada, que concentra a *toolbar*, no painel lateral da árvore de arquivos, no editor, nos terminais e na barra de status. Este capítulo apresenta cada região; o uso detalhado fica nos capítulos dedicados.

FIGURA 3 *Janela principal da AURORA com um projeto aberto*

Visão geral: toolbar no topo, árvore à esquerda, editor ao centro, terminais embaixo, barra de status no rodapé.

3.1 A barra de título e a toolbar

A barra superior organiza-se em três zonas. À esquerda ficam a identidade e o projeto: o ícone do SAPHO, o botão que mostra ou esconde a barra lateral e os botões Novo Projeto e Abrir Projeto.

O centro concentra a cadeia do processador, na ordem em que ela é usada. O Hub de Processadores cria processadores (Capítulo 6). O botão Compilar C±, com o glifo C±, compila o fonte até Verilog. A engrenagem ao lado abre as configurações de simulação do processador (*clock*, número de ciclos, exibição de *arrays*). O botão Sintetizar Verilog, com o ícone de pena, verifica os arquivos Verilog com o Icarus Verilog, e o PRISM abre o visualizador de RTL (Capítulo 16). Seguem dois seletores de estado global: o de simulador, que alterna entre Icarus Verilog e Verilator, e o de visualizador de ondas, entre GTKWave e Surfer. O botão Analisar Verilog, com o ícone de onda quadrada, roda a simulação e abre as formas de onda; o Fast Sim executa a simulação rápida, sem ondas, disponível apenas com o Verilator; o Cancelar interrompe todos os processos em andamento. Fecham a zona central o botão Configuração de ondas, que escolhe os sinais gravados na simulação, o seletor de arquivo `.gtkw` ativo e o botão de

Teste do processador sintetizado, com ícone de chip, que exercita as portas de entrada e saída do processador via Verilator.

À direita ficam os apoios e os controles de janela: o Controle de Versão (o painel Dagr, Capítulo 18), as Configurações do Aurora (Capítulo 19), o Assistente IA, com o ícone de estrela, que abre a Aurora Intelligence (Capítulo 17), e os botões de minimizar, maximizar e fechar. Um duplo clique na área vazia da barra também maximiza ou restaura a janela.

FIGURA 4 *Toolbar completa em close*

A barra de título inteira, com um projeto aberto e os botões habilitados; se possível, uma segunda versão com o tooltip de um botão visível.

3.2 O painel da árvore de arquivos

À esquerda fica o painel da árvore de arquivos, com o nome do projeto aberto junto ao rótulo. O mesmo painel abriga três visões, alternadas por um único botão no cabeçalho: a visão Arquivos, padrão, agrupada por processador e com cada Verilog no papel que você escolheu, sintetizável ou *testbench*; a visão Hierarquia, que mostra a árvore de instâncias de módulos extraída pelo Yosys após uma síntese; e a visão Pastas, um explorador de diretórios convencional, no estilo do VS Code, com operações completas de criação, renomeação e exclusão. O cabeçalho reúne ainda os botões de novo arquivo, atualização, busca nos arquivos (Ctrl+Shift+F), abertura no explorador do sistema, *backup* do projeto em `.zip`, recolhimento da árvore e fechamento do projeto. As três visões são detalhadas no Capítulo 4.

3.3 O editor

O centro da janela pertence ao editor de código, construído sobre o Monaco, o mesmo motor do VS Code: abas múltiplas, divisão em até três painéis, realce de sintaxe para C±, *assembly*, Verilog e SystemVerilog, diagnósticos em tempo real e formatação automática. O Capítulo 11 é dedicado a ele.

3.4 Os terminais

A área inferior reúne seis terminais em abas. Cinco deles, TCMM, TASM, TVERI, TWAVE e THTEST, são consoles de saída das etapas de compilação e simulação, cada etapa escrevendo no seu; o sexto, TCMD, é um terminal PowerShell interativo completo. Filtros por tipo de mensagem, exportação de log e números de linha clicáveis completam o conjunto (Capítulo 13).

3.5 A barra de status

No rodapé, da esquerda para a direita: o indicador Pronto ou Não Pronto, cujo ponto verde ou vermelho resume se o projeto está em condições de compilar, e o nome do processador ativo, deduzido do arquivo C± em foco no editor; ao centro, o botão Iniciar Compilação, com o ícone de raio; à direita, os avisos de estrutura (“Sem top-level”, “Sem testbench”), o motor de simulação selecionado, a posição do cursor e o indicador do GitHub.

3.6 Paleta de comandos, notificações e tooltips

A paleta de comandos, aberta com Ctrl+Shift+P, oferece busca difusa sobre os comandos da IDE, organizados em grupos; é o caminho mais curto quando se sabe o nome da ação, mas não onde fica o botão. Eventos como a criação de arquivos ou a troca de simulador aparecem como notificações discretas no canto da janela. Todo botão tem uma dica de ferramenta detalhada ao passar o mouse, e essas dicas podem ser desligadas nas Configurações.

3.7 A tela de boas-vindas

Sem projeto aberto, a AURORA mostra a tela de boas-vindas: a marca SAPHO com a *tagline* “Processador de Arquitetura Escalável para Otimização de Hardware”, os botões de criar e abrir projeto e a lista dos últimos dez projetos recentes, tudo sobre o fundo animado de aurora boreal.

PARTE II

Projetos e processadores

CAPÍTULO 4

Projetos

Tudo no SAPHO acontece dentro de um projeto: uma pasta no disco governada por um arquivo `.spf` (*SAPHO Project File*). O projeto reúne um ou mais processadores, os arquivos Verilog de integração e os metadados de que a AURORA precisa para orquestrar a compilação e a simulação.

4.1 Criando um projeto

Clique em Novo Projeto, na *toolbar* ou na tela de boas-vindas. O modal Criar Novo Projeto pede duas informações. O nome do projeto aceita apenas letras, números, sublinhado e hífen, sem espaços, acentos ou símbolos. O local é um campo somente leitura, preenchido pelo botão Procurar, que abre o seletor de pastas do Windows; a AURORA lembra o último local usado. Se algum campo violar as regras, o diálogo Entrada Inválida explica o problema antes de qualquer coisa ser criada.

Confirmado o formulário com Gerar Projeto, surge no disco a estrutura `<Local>/<Nome>/<Nome>.spf`. Um projeto recém-criado ainda não tem processadores; a árvore abre praticamente vazia, e o próximo passo natural é o Hub de Processadores (Capítulo 6). Para acompanhar o exemplo condutor deste manual, crie o projeto `MeuFiltro`.

FIGURA 5 *Modal Criar Novo Projeto preenchido*

Com o nome MeuFiltro e um local qualquer, antes de clicar em Gerar Projeto.

4.2 O arquivo `.spf`

O `.spf` é um arquivo JSON legível, com dois blocos. O bloco `metadata` guarda o nome do projeto, as datas de criação, modificação e última abertura, o nome do

computador e a versão da AURORA que o criou. O bloco `structure` descreve o conteúdo: a lista de processadores, as pastas, o arquivo *top-level*, o *testbench* e as listas de fontes Verilog sintetizáveis e de teste.

Você raramente precisará editá-lo à mão. A AURORA é a única escritora do arquivo e o mantém em dia a cada ação na interface. Como os caminhos são gravados relativos à pasta do projeto, o projeto inteiro pode ser copiado, movido de máquina ou versionado no Git sem quebrar; ao abrir, a AURORA realinha os caminhos ao novo local.

4.3 Abrindo um projeto

Há três caminhos: o botão Abrir Projeto, selecionando o `.spf`; dois cliques no próprio `.spf` pelo Explorer do Windows, graças à associação criada na instalação; ou a lista de recentes da tela de boas-vindas, que guarda até dez projetos e também alimenta a *jumplist* do ícone da AURORA na barra de tarefas. Ao abrir, a AURORA carrega a árvore, restaura a lista de processadores e registra a data de abertura. Se o `.spf` tiver sumido do caminho registrado, o diálogo Projeto Não Encontrado avisa, e o item sai da lista de recentes.

Projetos de versões antigas

Antes do YANC 7.1, cada processador tinha três subpastas, `Software`, `Hardware` e `Simulation`, e o nome do módulo vinha da diretiva `#PRNAME`. O compilador novo não aceita mais nenhuma das duas coisas. Ao abrir um projeto assim, a AURORA pergunta, uma vez, se pode convertê-lo. O que você escreveu (o fonte, os arquivos de vetor, os `input_<i>.txt`, um `.v` seu que estivesse em `Hardware`) vai para a pasta do processador; o que o YANC tinha gerado vai para a pasta de saída; a linha `#PRNAME` sai do fonte, e o `.spf` passa a apontar para os lugares novos. Nada é apagado nem sobrescrito: se o destino já existe, o arquivo fica onde estava e aparece no relatório.

No fim, um relatório lista o que foi movido e o que ficou. Se o `#PRNAME` antigo dava ao módulo um nome diferente do nome da pasta, o relatório oferece renomear o processador para esse nome, para o seu *top-level* continuar achando o módulo; o mesmo vale para um nome que não serve mais, com hífen ou acento. Quem responder Agora não pode converter depois, abrindo o projeto de novo; até lá ele não compila.

4.4 Arquivos do projeto: quem é quem

Um projeto maduro tem esta forma, aqui ilustrada com o nosso exemplo:

```
MeuFiltro/
|-- MeuFiltro.spf              (o arquivo de projeto)
```

```
|-- media_movel/              (um processador: o que voce escreve)
|   |-- media_movel.cmm
|   |-- input_0.txt
|   `-- media_movel/          (pasta de saida: media_movel.v,
|                              *.mif, media_movel_tb.v, ...)
`-- TopLevel/                 (Verilog de integracao, opcional)
```

A pasta de um processador guarda o que você escreve: o fonte, os arquivos que inicializam vetores e os estímulos `input_<i>.txt` (e os `cade_k.txt`, se o processador usar *links*; Seção 10.5). Dentro dela, a pasta de saída, com o mesmo nome, guarda o que o YANC gera; ela aparece na primeira compilação, e o Capítulo 12 lista o seu conteúdo.

Dois papéis especiais são atribuídos por você, pelo menu de contexto da árvore. O *top-level* é o módulo Verilog que integra os processadores; é o topo da síntese e o alvo do PRISM, definido por Definir como Top Level. O *testbench* é o banco de testes da simulação, um `_tb.v` em Verilog ou um `.py` em cocotb, marcado por Marcar como Testbench. A barra de status lembra o que falta em cada momento. Num projeto de um único processador, o *testbench* gerado automaticamente pela compilação costuma bastar; os papéis manuais importam quando você escreve a sua própria integração.

4.5 As três visões da árvore

O botão de alternância no cabeçalho do painel circula entre três visões do mesmo projeto.

A visão Arquivos é a padrão e a de trabalho: um seletor agrupado por processador, com os fontes C±, os Verilog separados entre sintetizáveis e *testbenches* e os artefatos relevantes. O papel de cada Verilog é você quem escolhe, pelo menu de contexto: um arquivo novo ou importado entra sem papel, com um ponto de interrogação no ícone, e fica fora de toda compilação até ganhar um. O que o YANC gera já nasce com o papel certo, o `<proc>.v` sintetizável e o `<proc>_tb.v` *testbench*.

A visão Hierarquia aparece depois de uma síntese (por exemplo, ao abrir o PRISM) e mostra a árvore de instâncias de módulos como o Yosys a enxerga, do *top-level* a cada instância aninhada.

A visão Pastas é um explorador de arquivos completo, com o menu de contexto cobrindo todo o ciclo: criar arquivo, pasta, fonte Verilog, *testbench* cocotb ou `.gitignore`; recortar, copiar, colar, renomear e excluir, com a escolha entre mover para a Lixeira, de onde se pode restaurar, e excluir permanentemente; copiar o caminho completo ou relativo; abrir a pasta no terminal integrado ou revelá-la no Explorador de Arquivos. Os nomes são validados como o Windows exige, sem caracteres proibidos nem nomes reservados, e conflitos de colagem oferecem substituir ou manter ambos.

FIGURA 6 *As três visões da árvore, lado a lado*

Três prints da mesma região: visão Arquivos, visão Hierarquia (após abrir o PRISM) e visão Pastas com o menu de contexto aberto.

> NOTA
>
> A árvore aceita a importação de arquivos `.v`, `.sv`, `.vh` e *testbenches* cocotb `.py` por arrastar e soltar. Outros tipos são recusados com uma dica; arquivos `.gtkw`, por exemplo, entram pelo seletor próprio da *toolbar*.

4.6 Manutenção do projeto

O botão Backup do projeto, no cabeçalho da árvore, gera sob demanda um `.zip` do projeto inteiro. Se arquivos registrados no `.spf` sumirem do disco, um aviso lista quantos faltam e oferece dispensá-los, removendo as referências. O botão vermelho de fechar projeto pede confirmação antes de agir. Renomear o projeto inteiro também é possível, inclusive pela Aurora Intelligence; a operação reescreve o `.spf` e a pasta em um só passo.

CAPÍTULO 5

O processador SAPHO

Antes de criar processadores na IDE, vale entender o que exatamente está sendo criado. Este capítulo descreve a arquitetura do *soft-processor* SAPHO do ponto de vista de quem o usa: o que os parâmetros significam, como a máquina executa o programa e quais são as suas restrições. Nada aqui precisa ser decorado para usar a plataforma, mas quem entende a máquina escreve programas melhores para ela.

5.1 Visão geral da arquitetura

O SAPHO é um processador de acumulador com arquitetura Harvard. Em vez de um banco de registradores, existe um único registrador central, o acumulador (ACC), que recebe o resultado de toda operação da unidade lógica e aritmética (ULA); o padrão dominante é buscar um operando na memória de dados, tomar o próprio acumulador como segundo operando e devolver o resultado ao acumulador, com uma pilha de dados fornecendo o segundo operando quando a expressão exige. A arquitetura Harvard, por sua vez, separa fisicamente as memórias de programa e de dados, cada uma com a sua largura e o seu barramento: o programa não pode se sobrescrever, e instruções e dados são acessados em paralelo.

Três traços definem o estilo da máquina. Primeiro, ela é totalmente parametrizável: toda largura é um parâmetro Verilog, da palavra de dados à mantissa e ao expoente do ponto flutuante, passando pelas profundidades das memórias e das pilhas. Segundo, vale o princípio de pagar apenas pelo que se usa: cada instrução do conjunto é um parâmetro booleano, o montador do YANC liga somente as instruções que o programa emprega, e o hardware das demais é eliminado na geração do circuito. Terceiro, o ciclo é curto e previsível: um *pipeline* raso de três estágios, busca, decodificação e execução, sem *forwarding*; como a ULA é combinacional e acumula a cada ciclo, nem os saltos condicionais quebram o fluxo, uma virtude e tanto em instrumentação.

5.2 Os parâmetros fundamentais

Ao criar um processador (Capítulo 6) você escolhe, direta ou indiretamente, os valores da Tabela 5.1. Eles aparecem como diretivas no topo do arquivo C± e como `parameter`

no Verilog gerado.

Tabela 5.1: Parâmetros de arquitetura do processador e as diretivas correspondentes.

Parâmetro	Diretiva	Significado
`NUBITS`	`#NUBITS`	Largura da palavra de dados, em bits; todo inteiro do programa tem esse tamanho, em complemento de dois.
`NBMANT`	`#NBMANT`	Largura da mantissa do ponto flutuante.
`NBEXPO`	`#NBEXPO`	Largura do expoente do ponto flutuante.
	`#NDSTAC`	Profundidade da pilha de dados.
	`#SDEPTH`	Profundidade da pilha de sub-rotinas.
	`#NUIOIN`	Número de portas de entrada.
	`#NUIOOU`	Número de portas de saída.
	`#NUGAIN`	Constante de divisão da função `norm()`.
	`#FFTSIZ`	Tamanho da FFT ($2^{\texttt{FFTSIZ}}$ pontos) no endereçamento com reversão de bits.

> AVISO
>
> A palavra precisa comportar exatamente o formato de ponto flutuante, isto é, `NUBITS` deve ser igual a `NBMANT` mais `NBEXPO` mais um (o bit de sinal). A AURORA valida essa igualdade no formulário de criação de processador e recusa combinações inválidas.

5.3 O ponto flutuante do SAPHO

O SAPHO tem hardware de ponto flutuante próprio, que não segue o padrão IEEE 754. O formato reúne um bit de sinal, `NBEXPO` bits de expoente em complemento de dois e `NBMANT` bits de mantissa com o bit líder explícito. A escolha permite precisões arbitrárias, como uma mantissa de 10 bits com expoente de 5, economizando hardware exatamente onde a aplicação permite.

Na prática, dentro do programa C± o tipo `float` simplesmente funciona: literais, aritmética e as funções da biblioteca operam nesse formato interno, e a conversão entre o formato IEEE do seu computador e o formato do SAPHO é feita em software pela cadeia de ferramentas, na carga das memórias e na leitura dos resultados. A precisão obtida é exatamente a configurada; com `#NBMANT 10`, espere cerca de três dígitos decimais significativos. Números complexos, o tipo `comp` da linguagem, não têm hardware dedicado: cada complexo é um par de `float`, parte real e imaginária, manipulado componente a componente pelo código gerado.

5.4 Como o programa vira circuito

O processador gerado compõe-se de poucos módulos Verilog, criados a partir de moldes parametrizáveis que acompanham o YANC. O módulo `processor` é o topo e instancia o núcleo e as duas memórias: `mem_instr`, a memória de programa, somente leitura, carregada com a imagem `<proc>_inst.mif`, e `mem_data`, a memória de dados, carregada com `<proc>_data.mif` e capaz de uma leitura e uma escrita por ciclo. O núcleo, `core`, reúne o caminho de dados com o acumulador, as pilhas, a busca de instruções e o contador de programa, apoiado pelo decodificador `instr_dec` e pela `ula`, na qual cada operação é um submódulo e somente os usados sobrevivem. Completam o conjunto o decodificador de endereços `addr_dec`, para múltiplas portas, e a FIFO genérica `myFIFO`, útil para acoplar o processador ao mundo externo sem depender de IP de fabricante.

O conjunto de instruções usa *opcode* de 7 bits, com famílias de carga e armazenamento (direto, indexado e com reversão de bits para FFT), pilha, entrada e saída, controle de fluxo e cálculo, este cobrindo a aritmética inteira e em ponto flutuante, a lógica, as comparações, os deslocamentos e as conversões. Essas instruções não são escritas à mão, o compilador C± as gera; mas você as verá nas formas de onda, pois a AURORA exibe, em sincronia com o *clock*, a instrução *assembly* e a linha C± correspondente a cada ciclo (Capítulo 15).

5.5 Entrada, saída e o mundo externo

O processador conversa com o exterior por portas numeradas de entrada e de saída, criadas conforme as diretivas `#NUIOIN` e `#NUIOOU` e acessadas no programa pelas funções `in()`, `out()`, `fin()` e `fout()` (Capítulo 9). No hardware, cada porta vira sinais de dados e de controle no topo do módulo `processor`; é por eles que o projeto de FPGA conecta o processador a conversores, FIFOs e outros blocos.

Dois recursos opcionais completam a interface. A interrupção faz o processador desviar para um ponto configurado do programa quando o pino correspondente pulsa. O marcador `#TOAQUI` gera um pino chamado `cheguei`, pulsado sempre que o programa passa pelo ponto marcado, o que permite sincronizar hardware externo com fases do algoritmo.

5.6 Um processador por tarefa

Como cada processador é gerado sob medida, o padrão de projeto no SAPHO difere do mundo dos processadores fixos: em vez de uma máquina grande que faz tudo, criam-se várias máquinas pequenas, uma por tarefa, conectadas por um *top-level* Verilog; um processador que detecta cruzamentos de zero pode alimentar outro que calcula uma distância, por exemplo. A AURORA suporta esse fluxo diretamente, pois

um projeto pode conter quantos processadores forem necessários, cada um com a sua pasta, o seu fonte e os seus parâmetros, todos integrados por um *top-level* e um *testbench* comuns (Capítulo 4).

CAPÍTULO 6

Criando processadores: o Hub de Processadores

Com o projeto aberto, o botão Hub de Processadores da *toolbar* abre o formulário que dá vida a um novo processador SAPHO. É aqui que as escolhas de arquitetura do Capítulo 5 viram campos concretos.

FIGURA 7 *Modal Criar Hub de Processadores*

O formulário completo, preenchido para o processador media_movel do exemplo condutor (Total de Bits 16, Mantissa 10, Expoente 5).

6.1 Os campos do formulário

O formulário pede o nome do processador, a linguagem do fonte (C± ou C++) e, em duas seções, os parâmetros numéricos. A Tabela 6.1 resume os campos, os valores de fábrica e as regras de validação; o texto a seguir comenta as escolhas.

A validação acontece enquanto você digita: um campo inválido ganha borda vermelha e uma dica explicando a regra, e o botão Gerar Processador só habilita quando tudo está consistente. Note que o padrão de fábrica já respeita a igualdade estrutural, pois $23 = 16 + 6 + 1$.

Sobre as escolhas: o total de bits dimensiona os inteiros e o consumo de hardware, e 16 bits bastam para muitos sinais de instrumentação, enquanto o padrão de 23 dá folga com um `float` de boa precisão. A mantissa e o expoente definem a precisão e a faixa do ponto flutuante (Seção 5.3); se o programa só usa inteiros, esses valores têm pouco efeito prático, mas a igualdade estrutural continua obrigatória. As pilhas não aparecem no formulário: sem `#SDEPTH` e `#NDSTAC` no fonte, o YANC dimensiona cada uma pelo programa, com o pico de uso mais um e no mínimo dois. Quem quiser um tamanho fixo escreve a diretiva à mão (Apêndice A). Reserve uma porta para

Tabela 6.1: Campos do Hub de Processadores, com os valores de fábrica e as validações.

Campo	Diretiva	Padrão	Validação
Nome do Processador	(arquivo)	`procTest_00`	letras sem acento, dígitos e sublinhado; sem dígito nem `__` no começo; nenhuma palavra reservada do Verilog
Total de Bits	`#NUBITS`	23	inteiro positivo, igual a mantissa + expoente + 1
Ganho	`#NUGAIN`	128	inteiro positivo, potência de 2; escrito só quando difere de 128
Bits da Mantissa	`#NBMANT`	16	inteiro positivo
Bits do Expoente	`#NBEXPO`	6	inteiro positivo
Portas de Entrada	`#NUIOIN`	1	inteiro positivo
Portas de Saída	`#NUIOOU`	1	inteiro positivo

cada fluxo de dados que o processador troca com o mundo, e lembre que o ganho só importa se o programa usa a função `norm()`.

6.2 O que é gerado

Ao confirmar, a AURORA cria dentro do projeto a pasta do processador, `<nome>/`, e registra o novo processador no `.spf`, bloqueando nomes duplicados. A pasta nasce só com o fonte, `<nome>/<nome>.cmm`; a pasta de saída, `<nome>/<nome>/`, para onde vai tudo o que o YANC gera, aparece na primeira compilação (Capítulo 12). O nome do processador é o nome do arquivo fonte e vira o nome do módulo Verilog; não há diretiva de nome. O `.cmm` nasce com o cabeçalho de diretivas preenchido com os valores do formulário e um `main()` vazio à sua espera:

```
#NUBITS 16
#NUIOIN 1
#NUIOOU 1
#NBMANT 10
#NBEXPO 5

void main()
{
    // Øk. Você criou um processador em C±, mas e agora?
}
```

E agora, o Capítulo 7: escrever o programa.

6.3 Configurações de simulação do processador

Ao lado do botão de compilar há uma engrenagem que abre um pequeno painel com três ajustes por processador: a frequência de *clock* do *testbench* gerado, em MHz (padrão 100); o número de ciclos que a simulação executa antes de encerrar (padrão 2000); e a opção de emitir cada elemento de *array* como um sinal visível nas formas de onda, equivalente à opção `--wave-arrays` do compilador. O painel também exibe o tempo estimado de simulação, os ciclos divididos pelo *clock*, em microssegundos. Se a simulação termina antes de o programa produzir os resultados, é aqui que se aumenta o número de ciclos.

6.4 Renomear e excluir processadores

Ambas as operações ficam no menu de contexto do processador na árvore. Renomear é uma operação completa: a pasta do processador, o fonte e a pasta de saída trocam de nome juntos, com os artefatos gerados que levam o nome dele, tudo numa só ação, sob as mesmas regras de nome do formulário. Excluir remove a pasta do processador e o registro no `.spf`.

> AVISO
>
> Como o nome do processador é o nome do arquivo fonte, renomear o `.cmm` muda o módulo Verilog gerado. Renomeie sempre pela AURORA, que mantém a pasta, o fonte, a pasta de saída e o `.spf` em sincronia.

PARTE III

A linguagem C±

CAPÍTULO 7

C±: fundamentos

A C± (lê-se "C mais-menos"; nos arquivos, extensão `.cmm`) é a linguagem principal do SAPHO: um dialeto de C criado pelo NIPS-CERN, enxuto o bastante para virar hardware eficiente e expressivo o bastante para algoritmos de processamento de sinais, com números complexos como tipo nativo e álgebra linear em notação de Dirac. Este capítulo apresenta a estrutura de um programa; os seguintes cobrem tipos, operadores, entrada e saída, biblioteca e recursos avançados.

> NOTA
>
> Tudo o que está nesta parte foi extraído da gramática oficial do compilador, nos arquivos `CMMComp.y` e `CMMComp.l` do repositório `nipscernlab/yanc`, cadeia de ferramentas na versão 5.3; o que a versão 7.1 acrescentou, os *links* entre processadores, está na Seção 10.5. Quando o manual diz que algo não existe na linguagem, é porque não existe na gramática, e o compilador rejeitará o código.

7.1 A estrutura de um programa

Um programa C± é uma sequência de três tipos de elemento, em qualquer ordem: diretivas, que são as linhas iniciadas por `#`; declarações de variáveis globais; e funções. O ponto de entrada é obrigatório, uma função `void main()`, e a sua ausência encerra a compilação com erro.

A forma canônica de um programa SAPHO coloca as diretivas no topo, configurando o processador, e em seguida o `main()` com um laço infinito `while (1)`; afinal, o programa modela um circuito que roda continuamente, processando amostras para sempre. O exemplo abaixo é o `media_movel`, o processador que acompanha o manual inteiro, um filtro de média móvel de quatro amostras, no arquivo `media_movel.cmm`:

```
#NUBITS 16
#NUIOIN 1
#NUIOOU 1
#NBMANT 10
#NBEXPO 5

void main()
{
```

```
    int x[4];    // historico das ultimas 4 amostras
    int soma;

    while (1)
    {
        x[3] = x[2];          // desloca o historico
        x[2] = x[1];
        x[1] = x[0];
        x[0] = in(0);         // le nova amostra da porta 0

        soma = x[0] + x[1] + x[2] + x[3];
        out(0, soma >> 2);    // media = soma/4, sem usar divisor
    }
}
```

O nome do processador não aparece no código: é o nome do arquivo. O `media_movel.cmm` gera o processador `media_movel` e o módulo Verilog `media_movel`.

Repare em duas escolhas típicas de quem escreve para o SAPHO. O histórico é um *array* deslocado à mão, e a divisão por quatro é um deslocamento de bits (`>> 2`); assim o processador gerado não precisa instanciar um divisor, que é um dos blocos mais caros da ULA.

7.2 Diretivas: configurando o processador no próprio código

As diretivas são linhas que começam com `#`, uma por linha, sem ponto e vírgula ao final. São elas que definem a arquitetura do processador a ser gerado; o nome vem do arquivo, e a antiga diretiva `#PRNAME`, se aparecer, é erro de compilação. Quando você cria um processador pela AURORA (Capítulo 6), o formulário preenche essas diretivas no `.cmm` inicial; a partir daí o arquivo é a fonte da verdade, e editar uma diretiva muda o processador na compilação seguinte. A referência completa, com os valores assumidos quando cada diretiva é omitida, está no Apêndice A; as mais usadas são `#NUBITS`, a largura da palavra, `#NBMANT` e `#NBEXPO`, o formato do ponto flutuante, e `#NUIOIN` e `#NUIOOU`, as quantidades de portas.

> AVISO
>
> Lembre da restrição estrutural: `NUBITS` deve ser igual a `NBMANT` mais `NBEXPO` mais um. No exemplo acima, $16 = 10 + 5 + 1$. A AURORA valida a igualdade ao criar o processador, mas quem edita as diretivas à mão responde por mantê-la.

Existem ainda duas diretivas comportamentais, que aparecem no meio do código e marcam pontos do programa: `#PRACA`, o ponto de retorno da interrupção, e `#TOAQUI`, o marcador refletido no pino `cheguei`. Ambas são explicadas no Capítulo 10.

7.3 Constantes com `#define`

A C± tem um pré-processador mínimo embutido: `#define NOME corpo` define uma constante simbólica, expandida onde o nome aparecer.

```
#define SCALE  3
#define OFFSET 7

void main()
{
    int x;
    while (1)
    {
        x = in(0);
        x = x + SCALE;
        out(0, x);
        x = x + OFFSET;
        out(0, x);
    }
}
```

Três limitações distinguem esse pré-processador do C tradicional: só existem *defines* de objeto, de modo que macros com argumentos, como `#define DOBRO(x) ((x)*2)`, não são aceitas; não há `#include` nem `#ifdef` no lado C±, embora o fluxo C++ tenha pré-processador completo (Capítulo 10); e o limite é de 256 *defines* por programa. O editor da AURORA reconhece os seus `#define` e os colore como constantes conforme você digita.

7.4 Comentários

Como em C: `//` comenta até o fim da linha, e `/* ... */` comenta blocos.

7.5 Onde o arquivo vive

Dentro de um projeto AURORA, o código de cada processador fica em `<projeto>/<processador>/<processador>.cmm`, ao lado dos arquivos que inicializam vetores e dos estímulos `input_<i>.txt`. A compilação põe tudo o que gera, o *assembly*, o Verilog, as imagens de memória e o *testbench*, na pasta de saída `<projeto>/<processador>/<processador>/` (Capítulo 12).

CAPÍTULO 8

C±: tipos, operadores e controle

8.1 Tipos de dados

A C± tem exatamente quatro palavras-chave de tipo. O `void` marca a ausência de tipo, no retorno de funções que não retornam nada. O `int` é o inteiro de `#NUBITS` bits, em complemento de dois. O `float` é o ponto flutuante no formato próprio do SAPHO, com mantissa de `#NBMANT` bits e expoente de `#NBEXPO` bits (Seção 5.3). E o `comp` é o número complexo, um par de `float` com as partes real e imaginária, tratado como tipo de primeira classe da linguagem.

Não existe um tipo de ponto fixo separado, papel que cabe ao `float` configurável, e tampouco existem `char`, `double`, `struct`, `union`, `enum` ou *strings*; um literal de texto só aparece na inicialização de *arrays* por arquivo, adiante.

8.1.1 Literais complexos e o `i` reservado

Um literal complexo escreve-se `a+bi`, como em matemática:

```
comp z;
z = 3.0 + 4.0i;
```

Por causa disso, o identificador `i` é reservado para a unidade imaginária, e usá-lo como nome de variável é erro de compilação. Adote `k`, `n` ou `idx` para contadores.

8.1.2 Conversões entre tipos

Misturar `int`, `float` e `comp` numa expressão ou atribuição é permitido, mas o compilador emite avisos de conversão implícita, por exemplo ao guardar um `float` num `int` com perda. Usar `float` ou `comp` como condição de `if` ou `while`, ou como índice de *array*, também gera aviso. Trate esses avisos como cheiro de projeto: conversões explícitas e índices inteiros deixam o hardware gerado mais barato e o comportamento mais óbvio.

8.2 Operadores

A Tabela 8.1 reúne os operadores da linguagem, cuja precedência segue a do C.

Tabela 8.1: Operadores da C±.

Grupo	Operadores
Aritméticos	`+ - * / %` (módulo, só inteiros) e o `-` unário
Bit a bit	`& | ^ ~` (inversão)
Deslocamentos	`« » >>` (à direita, preservando o sinal)
Relacionais	`< > <= >= == !=`
Lógicos	`&& || !`
Incremento	`++` (como comando ou em expressão, inclusive em elemento de *array*)

As ausências importam tanto quanto as presenças.

> AVISO
>
> Não existem em C±: o decremento `--`, os operadores compostos (`+=`, `-=`, `*=` e afins), o operador ternário `?:`, o *cast* explícito e os ponteiros; os símbolos `*` e `&` são apenas multiplicação e E bit a bit. Escreva `x = x - 1;` onde escreveria `x--;`.

8.2.1 Operadores custam hardware

Cada operador usado pela primeira vez no programa instancia o circuito correspondente na ULA do processador gerado, e o terminal TASM anuncia cada instância durante a compilação. Divisão e módulo são os blocos mais caros; os deslocamentos, os mais baratos. Daí o idioma do exemplo condutor, `soma >> 2` em vez de `soma / 4`. A aritmética de complexos aceita as quatro operações, cada uma expandindo para o conjunto de operações reais e imaginárias necessárias.

8.3 Estruturas de controle

Todas as estruturas clássicas do C existem, com a mesma sintaxe: o `if` com `else`; o `while` e o `do while`; o `for`, que o compilador reescreve internamente como um `while` equivalente; o `switch` com `case` e `default`, incluindo o *fall-through* real do C, em que a execução continua no próximo `case` na ausência de `break`; e `break`, `continue` e `return`, os dois primeiros válidos apenas dentro de laços.

```
// exemplo: saturador
int satura(int x, int lim)
{
```

```
    if (x > lim)  return lim;
    if (x < -lim) return -lim;
    return x;
}
```

8.4 Funções

As funções seguem a sintaxe do C, `tipo nome(tipo p1, tipo p2, ...)`, com chamadas por argumentos posicionais e retorno por `return`. O número de argumentos é conferido em cada chamada, e a contagem errada é erro de compilação; conversões de tipo em parâmetros e no retorno geram aviso. A profundidade de chamadas aninhadas é limitada pela pilha de sub-rotinas, a diretiva `#SDEPTH`.

Duas restrições merecem destaque. *Arrays* não podem ser passados como parâmetro; trabalhe com *arrays* globais ou no próprio `main()`. E a recursão não é suportada no fluxo C±: cada variável, incluindo parâmetros e locais, vive num endereço fixo da memória de dados, de modo que uma função que chamasse a si mesma sobrescreveria os próprios dados. É justamente esse endereçamento fixo que permite à AURORA mostrar cada variável por nome nas formas de onda. Se o algoritmo é essencialmente recursivo, o caminho C++ atende (Capítulo 10).

```
int add(int a, int b)
{
    return a + b;
}

int triple_sum(int x, int y, int z)
{
    return x + y + z;
}

void main()
{
    while (1)
    {
        int v = in(0);
        out(0, add(v, 1));
        out(0, triple_sum(v, 1, 2));
    }
}
```

8.5 Arrays

A linguagem aceita *arrays* de uma e duas dimensões, com tamanho fixo, e uma forma de inicialização peculiar e muito útil:

```
int  x[128];               // sem inicializacao
int  h[8]    "coefs.txt";  // inicializado por arquivo, em tempo de
    compilacao
int  m[8][8] "matriz.txt"; // 2D, tambem por arquivo
```

A inicialização por arquivo lê os valores de um arquivo de texto ao lado do fonte e os grava diretamente na imagem da memória de dados do processador: os coeficientes do seu filtro já nascem na memória, sem custo de inicialização em tempo de execução.

Há duas formas de indexação. A normal, `x[k]`, e a bit-reversa, `x[k)`, que se distingue pelo fecha-parênteses: o índice é lido com os bits invertidos, o endereçamento clássico da FFT (*Fast Fourier Transform*), com a quantidade de bits invertidos definida pela diretiva `#FFTSIZ` (Capítulo 10). Elementos de *array* aceitam o incremento, como em `hist[k]++;`.

8.6 Variáveis e escopo

Declarações podem aparecer no topo do programa ou dentro de funções e blocos. Na prática, porém, todo nome vive num endereço fixo e único da memória de dados, e declarar duas variáveis com o mesmo nome é erro. Pense nas variáveis C± como registradores nomeados do seu circuito: é exatamente isso que elas se tornam, e é por isso que cada uma aparece por nome na forma de onda da simulação.

CAPÍTULO 9

C±: entrada, saída e biblioteca padrão

9.1 Entrada e saída

O processador conversa com o mundo por portas numeradas, criadas pelas diretivas `#NUIOIN` e `#NUIOOU`, e todo o tráfego passa por quatro funções intrínsecas: `in(p)` lê um inteiro da porta de entrada `p`; `fin(p)` lê convertendo para `float`; `out(p, expr);` escreve o valor da expressão na porta de saída `p`; e `fout(p, expr);` escreve em formato `float`.

O número da porta é um literal inteiro, validado contra as diretivas: usar `in(2)` num processador com `#NUIOIN 1` é erro de compilação. No hardware, `in()` é uma leitura com *handshake*, na qual o processador sinaliza `req_in` e aguarda o dado, e `out()` pulsa `out_en` com o dado estável no barramento, o suficiente para acoplar FIFOs e conversores. No nosso `media_movel`, `x[0] = in(0);` lê a amostra e `out(0, soma >> 2);` publica a média: uma porta de cada lado basta.

9.2 Funções especiais

Cinco funções existem para economizar hardware, cada uma evitando um bloco caro da ULA ou uma sequência de instruções. A `norm(x)` divide pelo valor da diretiva `#NUGAIN`, uma potência de dois, e entrega a normalização sem instanciar o divisor completo; funciona só com inteiros. A `pset(x)` devolve `x` quando positivo e zero quando negativo, o equivalente de `if (x<0) x=0;` numa única instrução. A `abs(x)` é o valor absoluto e, sobre um complexo, a magnitude. A `sign(x, y)` devolve `y` com o sinal de `x`. E a `copy(x, y);` copia os bits de `x` em `y` sem conversão nem checagem de tipo, uma reinterpretação crua, a ser usada com consciência.

9.3 Funções não lineares e de arredondamento

A biblioteca cobre raiz e trigonometria (`sqrt`, `sin`, `cos`, `tan`, `atan`), as hiperbólicas (`sinh`, `cosh`, `tanh`), exponencial e logaritmo (`exp`, `log`, `pow`) e os arredondamentos `floor`, `ceil` e `round`, que retornam `float`, com o `round` arredondando o meio para longe do zero.

Essas funções não são circuitos dedicados: são macros de *assembly* otimizadas, como o método de Newton na `sqrt` e séries nas trigonométricas, injetadas no programa quando usadas. Elas custam instruções e ciclos, não blocos de ULA, exceto pelas operações de ponto flutuante que elas próprias empregam. A `pow(x, y)` merece nota: com expoente inteiro constante, o compilador gera a sequência mínima de multiplicações; com expoente inteiro variável, um laço em tempo de execução; nos demais casos, a identidade $x^y = e^{y \ln x}$.

9.4 Funções para complexos

Seis funções operam sobre o tipo `comp`: `real(z)` e `imag(z)` extraem as partes; `fase(z)` devolve o argumento; `mod2(z)` devolve a magnitude ao quadrado, $a^2 + b^2$, mais barata que `abs` por dispensar a raiz; `complex(re, im)` monta um complexo a partir de dois `float`; e `conj(z)` devolve o conjugado.

```
comp x; comp y; comp r;
x = 1.0 + 2.0i;
y = 3.0 + 4.0i;
r = x * y;
fout(0, real(r));
fout(0, imag(r));
```

> AVISO
>
> Nem tudo se aplica a `comp`: `sqrt`, `exp`, `log`, `pow`, `sign` e `pset` sobre complexos são erro de compilação, assim como o incremento `z++`. O módulo `%` e a `norm()` são exclusivos de inteiros.

9.5 Álgebra linear em notação de Dirac

O recurso mais idiossincrático da C± é a escrita de operações vetoriais e matriciais em notação *bra-ket*, familiar a quem vem da física. Sendo `a` e `b` vetores (*arrays*) e `M` uma matriz, o produto interno é a expressão `<a|b>`; o comando `a # |0>;` zera o vetor; `a # |M|b>;` calcula o produto matriz-vetor; `a # c|b>;` escala um vetor por uma constante; `A # |a><b|;` forma o produto externo; e `out(p, c|a>);` emite o vetor escalado por uma porta de saída.

Essas construções expandem para os laços e instruções indexadas adequados, e são a forma idiomática de escrever filtros FIR, correlações e projeções em C±. Um FIR direto, por exemplo, é uma linha, `y = <h|x>;`, com `h` guardando os coeficientes, inicializados por arquivo, e `x` a janela de amostras.

CAPÍTULO 10

C±: recursos avançados

10.1 Interrupção: `#PRACA`

O processador SAPHO tem um pino de interrupção, o `itr`. Quando ele pulsa, o contador de programa desvia para o ponto do código marcado pela diretiva `#PRACA`, que, diferentemente das diretivas de configuração, aparece no meio do programa, marcando a linha de retorno. O uso típico é reiniciar o laço de processamento quando o hardware externo sinaliza um novo evento, como um gatilho de aquisição, sem esperar o programa completar a volta.

10.2 Sincronização com o hardware: `#TOAQUI`

A diretiva `#TOAQUI`, colocada num ponto do programa, faz o pino `cheguei` do processador pulsar sempre que a execução passa por ali. É um farol para o mundo externo, útil para sincronizar blocos de hardware com fases do algoritmo, e é também um mecanismo interno da própria AURORA: no teste de hardware do terminal THTEST e nas simulações com o Verilator, a IDE insere um `#TOAQUI` ao final do `main()` para detectar o término do programa e encerrar a simulação no instante certo.

10.3 FFT e o endereçamento bit-reverso

A diretiva `#FFTSIZ n` configura o circuito de reversão de bits para transformadas de 2^n pontos, usado pela indexação `x[k]` apresentada no Capítulo 8. Na FFT radix-2 os resultados saem em ordem bit-reversa; com `x[k]` a reordenação é gratuita, pois o hardware inverte os bits do índice na própria leitura ou escrita, sem instruções extras. O repositório do YANC traz um projeto de exemplo completo, o `proc_fft`, com uma FFT inteira em C±.

10.4 Quanto custa cada construção

O princípio de pagar apenas pelo que se usa tem uma consequência prática: o custo em FPGA do processador é proporcional ao subconjunto da linguagem que o programa emprega. Durante a compilação, o terminal TASM informa cada bloco de hardware instanciado à medida que os *opcodes* aparecem pela primeira vez. Chamar uma função instancia a memória da pilha de sub-rotinas; usar *arrays* instancia o circuito de endereçamento indexado, e a forma `x[k]`, o de reversão de bits; a divisão instancia o divisor inteiro, o `%` o módulo, e as versões em ponto flutuante, os blocos correspondentes. Ao final, o TASM resume o percentual da ISA (*Instruction Set Architecture*) e da ULA efetivamente usados pelo programa.

Daí algumas regras de bolso para hardware enxuto: prefira deslocamentos a divisões por potências de dois, ou use `norm()`; prefira `mod2()` a `abs()` quando só compara magnitudes; evite `float` quando a dinâmica do sinal cabe em inteiros; e lembre que é a primeira ocorrência de um operador que paga o bloco, pois a segunda em diante reaproveita o circuito.

10.5 Links entre processadores

Dois processadores SAPHO de um mesmo projeto podem trocar valores diretamente, por *links* numerados a partir de 0:

```
toma(0, x);        // entrega x pelo link 0 e espera o outro lado pegar
y = cade(0);       // espera o link 0 oferecer um valor e o recebe como int
f = fcade(1);      // o mesmo, recebendo como float
```

O número do *link* é sempre um literal inteiro, como a porta de `in(0)`. Quem chega primeiro espera, e nada se perde nem é lido duas vezes; um impasse, com os dois lados esperando, é responsabilidade do programa. Um processador tem quantos *links* o programa usar, e cada *link* liga dois processadores e pode levar valores nos dois sentidos. No *assembly*, as funções viram as instruções `TOM k` e `CAD k`, cujo operando é o número do *link*.

As formas `toma_sync`, `cade_sync` e `fcade_sync` são para quando o parceiro roda no mesmo *clock*: dispensam o sincronizador do *link* e economizam 4 ciclos por valor. Com *clocks* diferentes, use as formas simples. Um *link* usa a mesma forma nas duas pontas, e misturar as duas no mesmo *link* é erro de compilação. O compilador anota no *assembly*, com a diretiva `#LKSYNC k`, quais *links* usam a forma `_sync`; ela é escrita pelo compilador, não pela pessoa. O mesmo vale em C++.

A palavra passa sem conversão: os dois processadores precisam do mesmo `#NUBITS` e, para `float`, do mesmo `#NBMANT` e `#NBEXPO`. O compilador não confere isso, porque compila um programa de cada vez.

Cada *link* cria pinos no processador: `toma_k` e `valeu_k` em quem usa `toma(k,`

`...`), `cade_k`, `taqui_k` e `cade_dado_k` em quem usa `cade(k)`, e um `toma_dado` em quem usa algum `toma`. No *top-level* do projeto, o processador A (*link* i) liga ao B (*link* j) assim: `A.toma_i` em `B.taqui_j`, `B.cade_j` em `A.valeu_i` e `A.toma_dado` em `B.cade_dado_j`.

Simulado sozinho, o processador ganha do *testbench* gerado o papel do parceiro de cada *link*: os valores de cada `cade(k)` vêm de `cade_k.txt`, ao lado do fonte, um por linha, e cada `toma(k, x)` é escrito em `toma_k.txt`, na pasta de saída.

10.6 O caminho C++

Além da C±, o YANC compila C++ para o mesmo processador, pelo compilador CPPCOMP, que aceita um subconjunto de C++ com classes, `for`, referências e parte da biblioteca padrão, e emite o mesmo *assembly* intermediário. Ele mesmo faz o pré-processamento, com `#include`, `#define` completo, compilação condicional (`#if`) e `#pragma once`, e as mensagens de erro apontam o arquivo e a linha que você escreveu, inclusive dentro de um *header*. O processador C++ segue a regra da C±: o nome é o do arquivo, e `filtro.cpp` gera o módulo `filtro`.

O caminho C++ oferece o que a C± não tem: ponteiros, restritos à indexação de *arrays* de tamanho fixo e sem alocação dinâmica; recursão, com quadros de pilha de verdade; *structs* simples; e até recursos modernos como *lambdas* e destrutores. Os cabeçalhos padrão adaptados incluem `array`, `cmath`, `cstdint` e `vector`. O preço é a visibilidade: variáveis locais deixam de ter endereço fixo e, com isso, deixam de aparecer por nome nas formas de onda, perdendo-se em parte o vínculo íntimo entre código e onda que marca o fluxo C±. Os parâmetros do processador nesse fluxo têm padrões próprios, palavra de 32 bits com mantissa de 23 e expoente de 8, ajustáveis por `#pragma yanc`.

> NOTA
>
> Recomendação prática: comece todo projeto em C± e migre para C++ apenas quando precisar especificamente de recursão, ponteiros ou *structs*. Mantenha em C± os processadores de sinal críticos, nos quais a visibilidade de simulação é total.

10.7 As mensagens do compilador

Os diagnósticos do YANC são bilíngues, acompanhando o idioma da IDE (Capítulo 19); as mensagens do CPPCOMP saem sempre em inglês. Os do CMMCOMP têm personalidade: um erro de sintaxe rende um comentário bem-humorado, e esquecer o `main()` provoca a pergunta clássica sobre onde ele está. Por trás do humor, a informação é séria e cai em poucas classes. Erros de estrutura cobrem a falta do `main()`, o `break` ou `continue` fora de laço e a contagem errada de parâmetros. Erros de nome apontam variável inexistente, variável já declarada, o uso do `i` reservado e o

arquivo cujo nome não serve de nome de módulo Verilog; a diretiva `#PRNAME`, que não existe mais, também é erro. Erros de tipo cobrem o módulo e a `norm()` com não inteiros, os intrínsecos sem versão para complexos e o incremento de um `comp`. Erros de faixa denunciam o estouro de inteiro para o `#NUBITS` escolhido, o `float` fora da faixa do formato configurado e a porta de entrada ou saída inexistente. Os avisos, por fim, sinalizam conversões implícitas entre tipos e o uso de `float` ou `comp` em condições e índices.

Os erros aparecem no terminal TCMM com a linha clicável, e o clique leva o editor ao ponto exato (Capítulo 13). A Aurora Intelligence também sabe explicar qualquer mensagem do compilador, pela ferramenta de consulta embutida (Capítulo 17).

PARTE IV

O editor

CAPÍTULO 11

O editor

O editor da AURORA é construído sobre o MONACO, o mesmo motor de edição do Visual Studio Code. Multi-cursor, minimapa, localizar e substituir, desfazer ilimitado e a ergonomia geral que você já conhece vêm de fábrica; este capítulo cobre o que a AURORA acrescenta por cima.

11.1 Abas

Cada arquivo aberto vira uma aba na barra superior do editor, com um ponto indicando alterações não salvas. As abas podem ser reordenadas por arraste. Salva-se a aba atual com Ctrl+S e todas com Ctrl+Shift+S; fecha-se a atual com Ctrl+W, reabre-se a última fechada com Ctrl+Shift+T, e Ctrl+N cria um arquivo novo.

11.2 Editor dividido

O botão de divisão, que flutua no canto do painel em foco, abre o arquivo atual num novo painel lado a lado, até o limite de três painéis, cada um com a sua barra de abas. O detalhe importante é que painéis exibindo o mesmo arquivo compartilham o mesmo documento: editar de um lado atualiza o outro ao vivo, com desfazer e estado de salvamento unificados. É a forma natural de manter o `.cmm` à esquerda e o `.v` gerado à direita, ou duas regiões distantes do mesmo arquivo.

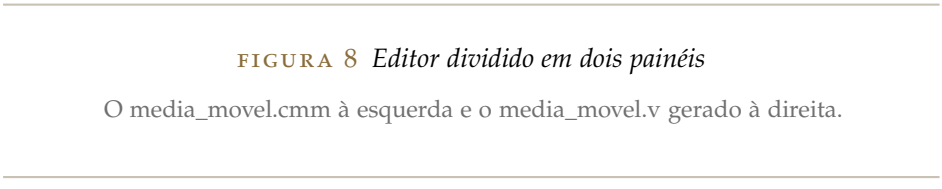


FIGURA 8 *Editor dividido em dois painéis*

O media_movel.cmm à esquerda e o media_movel.v gerado à direita.

11.3 Realce de sintaxe

A C± tem gramática de realce própria, cobrindo palavras-chave, diretivas, funções da biblioteca, literais complexos e a notação de Dirac; os `#define` do programa são recolhidos dinamicamente e coloridos como constantes. O *assembly* do SAPHO também tem gramática própria. O Verilog e o SystemVerilog usam a gramática do Monaco, enriquecida com *tokens* semânticos do tree-sitter, que igualmente atende C e C++ no caminho alternativo de compilação.

11.4 Diagnósticos e navegação

Dois servidores de linguagem (LSP, *Language Server Protocol*) trabalham nos arquivos Verilog. O VERIBLE é o servidor completo: diagnósticos de sintaxe e estilo enquanto se digita, formatação, *outline* de símbolos, *hover*, ir à definição e referências. O SLANG complementa com a análise semântica do projeto inteiro, elaborando todos os módulos em conjunto para apontar erros de elaboração e portas incompatíveis, além de oferecer autocompletar; por ser mais pesado, pode ser ligado e desligado com Ctrl+Alt+S. Os problemas aparecem sublinhados no código, e tudo funciona em regime de melhor esforço: sem um servidor disponível, a edição segue sem ele.

11.5 Formatação automática

O atalho Shift+Alt+F formata o documento atual, para C, C++ e C±, com o CLANG-FORMAT empacotado. Se o projeto tiver um arquivo `.clang-format` próprio, ele é respeitado.

11.6 Busca no projeto

O atalho Ctrl+Shift+F, ou o botão de busca no cabeçalho da árvore, abre o modal Buscar nos Arquivos: termo com opções de diferenciar maiúsculas, palavra inteira e expressão regular, resultados agrupados por arquivo e clique levando à linha. A busca ignora o `.git` e as pastas de artefatos.

11.7 Seleção e IA

Ao selecionar um trecho de código, um pequeno botão em forma de estrela aparece junto à seleção: ele envia o trecho diretamente à Aurora Intelligence, já contextualizado. É o caminho mais curto para pedir uma explicação ou perguntar por que algo não compila (Capítulo 17).

11.8 Conflitos com o disco

Se um arquivo aberto for alterado por outro programa, a AURORA detecta a divergência e pergunta o que fazer no diálogo Arquivo Modificado Externamente, oferecendo manter a versão do editor, usar a versão do disco ou salvar e recarregar. Nada é sobrescrito sem a sua decisão.

11.9 Atalhos de teclado

Os atalhos citados neste capítulo, todos reconfiguráveis nas Configurações, estão reunidos com os demais no Apêndice C. Dentro do editor valem ainda os atalhos usuais do Monaco, como Ctrl+F para localizar, Alt+Click para múltiplos cursores e Ctrl+/ para comentar a linha. A posição do cursor é exibida na barra de status.

PARTE V

Compilação e simulação

CAPÍTULO 12

Compilação

Com o `media_movel.cmm` escrito, é hora de transformá-lo em hardware. Este capítulo explica o que acontece quando cada botão de compilação é acionado e quais arquivos aparecem no projeto.

12.1 O pipeline do YANC em duas etapas

Todo clique em Compilar C± executa, por processador, os dois compiladores do YANC. A AURORA chama um programa só, o YANC, uma vez para cada etapa, e ele chama os compiladores que estão ao lado dele. Na primeira etapa, o CMMCOMP traduz o `<proc>.cmm` em *assembly* simbólico, o `<proc>.asm`, junto com os registros e as tabelas que ligam cada instrução à linha do fonte; um fonte `.cpp` passa pelo CPPCOMP, que produz o mesmo `.asm` (Seção 10.6). Na segunda, o ASMCOMP monta o *assembly* em duas passadas: a primeira conta instruções e dados e resolve os endereços de variáveis e rótulos; a segunda gera o processador em Verilog (`<proc>.v`), as duas imagens de memória, `<proc>_inst.mif` com o programa e `<proc>_data.mif` com os dados, e o *testbench* (`<proc>_tb.v`). Qual compilador roda na primeira etapa, a AURORA decide pela extensão do fonte.

O terminal TCMM mostra a saída da primeira etapa; o TASM, a da segunda, incluindo os avisos de recurso instanciado que revelam o custo de hardware do código (Seção 10.4).

> INFORMAÇÃO
>
> O processador gerado instancia os moldes de HDL do SAPHO que acompanham a instalação: o núcleo, a ULA, as memórias. O programa não é um arquivo carregado depois; as imagens `.mif` nascem embutidas no projeto de hardware, e reprogramar significa recompilar e sintetizar de novo.

12.2 Os botões de compilação

O botão Compilar C± roda o pipeline completo da seção anterior para os processadores do projeto, do fonte ao Verilog, memórias e *testbench*. O Sintetizar Verilog verifica os arquivos Verilog com o Icarus Verilog, apontando erros de sintaxe e elaboração no

terminal TVERI, com as linhas clicáveis. O Analisar Verilog percorre o fluxo completo até as formas de onda: compila o que for preciso, roda a simulação e abre o visualizador (Capítulo 14). O Fast Sim simula sem gravar ondas, disponível com o Verilator selecionado, para quando só interessam as saídas em arquivo. O PRISM sintetiza com o Yosys e abre o diagrama de RTL (Capítulo 16), exigindo um *top-level* definido. O Teste do processador sintetizado constrói um *harness* C++ com o Verilator em torno do `<proc>.v` e exercita apenas as portas de entrada e saída, o teste de fumaça do processador como caixa-preta, com progresso no THTEST. E o Cancelar interrompe todos os processos de compilação e simulação em andamento, sem fechar a IDE.

O botão Iniciar Compilação, o raio da barra de status, é um atalho para o fluxo principal, e todos esses comandos também existem na paleta (Ctrl+Shift+P), no grupo de compilação.

Antes de compilar, a AURORA salva os arquivos automaticamente. O fluxo de ondas exige um *testbench*, papel que o `<proc>_tb.v` gerado cumpre, e o PRISM exige um *top-level*; a barra de status avisa o que falta, e o processador ativo acompanha o `.cmm` em foco no editor.

12.3 Onde cada artefato aparece

Cada processador tem duas pastas: a dele, com o que você escreve, e dentro dela a pasta de saída, com o mesmo nome, onde o YANC põe tudo o que gera. Depois de compilar e simular o `media_movel`, a pasta dele fica assim:

```
media_movel/
|-- media_movel.cmm            (seu fonte)
|-- input_0.txt                (estimulo da porta de entrada 0)
`-- media_movel/               (pasta de saida: tudo o que o YANC gera)
    |-- media_movel.asm        (assembly gerado)
    |-- media_movel.v          (o processador em Verilog)
    |-- media_movel_inst.mif   (imagem da memoria de programa)
    |-- media_movel_data.mif   (imagem da memoria de dados)
    |-- media_movel_tb.v       (testbench gerado)
    |-- prog_log.txt, src_lines.txt, pc_media_movel_mem.txt,
    |   trad_opcode.txt, app_log.txt  (tabelas p/ as ondas)
    `-- output_0.txt           (o que a simulacao escreveu)
```

O nome de tudo vem do nome do arquivo fonte: `media_movel.cmm` gera o processador `media_movel` e o módulo Verilog `media_movel`. Não escreva nada dentro da pasta de saída, porque a próxima compilação sobrescreve. Um arquivo presente nela também não prova que a última compilação passou, pois pode ser sobra de uma tentativa anterior: a prova é o terminal, com as duas etapas terminando sem erro.

O `.v` e os `.mif` da pasta de saída são exatamente o que se leva ao Quartus ou ao Vivado na hora de sintetizar para o FPGA real. O Verilog gerado cita os `.mif`

pelo nome puro, sem pasta; na síntese, copie-os para a pasta do projeto do Quartus ou do Vivado, ou escreva o caminho completo nos parâmetros `DFILE` e `IFILE`. O *testbench* gerado produz *clock* e *reset*, alimenta o processador com os estímulos de `input_<i>.txt`, um valor por linha e um arquivo por porta de entrada, lidos da pasta do processador, um nível acima, e grava as saídas em `output_<i>.txt` na pasta de saída, além do despejo de ondas. Ele roda com a pasta de saída como diretório de trabalho, e a AURORA cuida disso.

> NOTA
>
> Segurança de execução: a AURORA só executa binários da própria cadeia de ferramentas, validados contra uma lista fechada, e os botões não passam por um *shell*. É por isso que a IDE não oferece a execução de comandos arbitrários fora do terminal TCMD.

CAPÍTULO 13

Os terminais

A área inferior da janela reúne seis terminais em abas. Cinco são consoles de saída, cada etapa da cadeia escrevendo no seu, o que torna previsível onde procurar cada mensagem; o sexto é um *shell* interativo de verdade. Quando você dispara uma ação na *toolbar*, a AURORA troca automaticamente para a aba certa conforme as etapas avançam.

FIGURA 9 *Área de terminais com o TCMM ativo após uma compilação*

Mostrar as seis abas, alguns cartões de mensagem e os filtros do topo.

13.1 Quem escreve onde

O TCMM recebe a saída do compilador C± ou do C++: erros, avisos e o progresso da tradução para *assembly*. O TASM recebe a do montador, a geração do Verilog e das memórias e os avisos de recurso instanciado. O TVERI concentra o Icarus Verilog e a síntese do Yosys, com os diagnósticos de Verilog. O TWAVE acompanha a execução da simulação, seja pelo vvp, pelo Verilator ou pelo cocotb, e a abertura dos visualizadores de onda. O THTEST é dedicado ao teste de entrada e saída do processador sintetizado, com barra de progresso própria. E o TCMD é o *shell* integrado, descrito na Seção 13.3.

13.2 Cartões, filtros e o modo verboso

As mensagens chegam como cartões classificados em erros, avisos, sucessos e dicas, no mesmo código de cores da IDE. No topo da área ficam os filtros por tipo, com contadores, que atuam quando o Modo Verboso está desligado nas Configurações; com o modo ligado, que é o padrão, tudo aparece. Ali também estão os botões de

limpar o terminal atual, exportar o conteúdo para arquivo e recarregar a interface. Cada terminal retém até cinco mil entradas.

Erros que citam arquivo e linha, como `arquivo.v:42:` do Icarus ou "linha 17" do C±, viram *links*: o clique abre o arquivo no editor, na linha exata. É o caminho mais rápido do erro à correção.

13.3 TCMD: o shell integrado

A aba TCMD é um terminal PowerShell real, com um *prompt* próprio da AURORA. Digite qualquer comando, como `git`, `python` ou `cd`, e tecle Enter; o diretório atual e as variáveis de ambiente persistem entre comandos, como num terminal comum. O menu de contexto da árvore de arquivos oferece Abrir no Terminal Integrado, que abre o TCMD já na pasta escolhida, e é também esse *shell* que a Aurora Intelligence usa quando você autoriza a ferramenta de execução de comandos (Capítulo 17).

> NOTA
>
> Use o TCMD para tarefas auxiliares, como Git avançado ou *scripts* Python que analisem os `output_*.txt`. Para compilar e simular, prefira os botões da *toolbar*, que cuidam de argumentos, caminhos e ordem das etapas por você.

CAPÍTULO 14

Simulação

Simular é executar o processador gerado, ciclo a ciclo, num simulador de Verilog, vendo cada variável, cada instrução e cada porta no tempo. A AURORA orquestra tudo com um clique em Analisar Verilog; este capítulo explica as escolhas disponíveis e como controlá-las.

14.1 Dois motores: Icarus Verilog e Verilator

O seletor de simulador da *toolbar* alterna o motor global. O Icarus Verilog, padrão, interpreta o Verilog, com o `iverilog` compilando e o `vvp` executando; é o motor da depuração, porque expõe os sinais internos do processador, das variáveis ao acumulador e ao contador de programa. O Verilator converte o projeto em C++ e compila um executável nativo, cerca de cinco a dez vezes mais rápido em simulações longas, mas expõe apenas os sinais do topo do *testbench*. Ao alternar, uma notificação resume a troca. Para o dia a dia de depuração do `media_movel`, fique no Icarus; quando quiser rodar milhões de ciclos, mude para o Verilator.

14.2 Dois tipos de testbench: Verilog e cocotb

O *testbench* determina quem estimula o processador. O de Verilog, o `_tb.v` gerado pela compilação ou um escrito por você, cria *clock* e *reset*, alimenta cada porta de entrada com o conteúdo de `input_<i>.txt`, um valor por linha, lido da pasta do processador, ao lado do fonte, e grava cada porta de saída em `output_<i>.txt`, na pasta de saída. Os *links* entre processadores seguem a mesma regra: `cade_k.txt` ao lado do fonte, `toma_k.txt` na pasta de saída (Seção 10.5). A simulação roda com a pasta de saída como diretório de trabalho. O de Python usa o *framework* cocotb: marque um arquivo `.py` como *testbench* pelo menu de contexto da árvore, que também oferece a criação de um novo, e a AURORA o executa com o Python embutido sobre o motor selecionado, o que convém a estímulos complexos, checagens automáticas e comparação com modelos NumPy.

Os dois tipos cruzam com os dois motores, formando quatro caminhos que convergem no mesmo arquivo de ondas. A simulação roda uma única vez por clique.

14.3 Parâmetros da simulação

O painel de configurações do processador (Seção 6.3) controla a frequência de *clock*, o número de ciclos e a exibição de *arrays* nas ondas, além de estimar o tempo simulado. Quando a saída do programa não aparece, a causa mais comum é simulação curta demais; aumente ali o número de ciclos.

14.4 Escolhendo os sinais das ondas

Por padrão, o despejo grava os sinais do escopo do *testbench*. O botão Configuração de ondas abre o modal Sinais para Dump, que lista todos os sinais descobertos no projeto, com busca por texto ou expressão regular, os botões de selecionar tudo, limpar e restaurar o padrão, um filtro para mostrar apenas sinais do processador e, no rodapé, o contador de selecionados. Menos sinais significam arquivos menores e simulação mais leve, o que pesa em execuções longas. Com o Verilator selecionado a regra é por escopo, e o modal avisa isso: o Verilator ignora os argumentos do `$dumpvars`, então a AURORA traduz a seleção em regras de escopo no arquivo de configuração do simulador. Todo sinal público de um módulo que tenha ao menos um sinal selecionado é gravado, e módulos sem nenhum sinal selecionado ficam fora do arquivo de ondas; o escopo do *testbench* é sempre gravado. Para reduzir o arquivo num projeto grande, portanto, o que conta é em quais módulos há sinais escolhidos. A mesma tradução vale para um *testbench* que traz os próprios `$dumpvars`, como o gerado pelo YANC, cujas chamadas sinal a sinal viram regras por módulo, para o padrão sem seleção, que grava só o escopo do *testbench* como no Icarus, e para *testbenches* em Python com o cocotb, em que a seleção entra pelos argumentos de build do Verilator.

FIGURA 10 *Modal Configuração de Ondas*

Com alguns sinais do media_movel filtrados e selecionados.

Na hora de gravar, a AURORA decide a lista de sinais por uma ordem de precedência: primeiro um `.gtkw` ativo no seletor da *toolbar*, cujos sinais referenciados mandam; depois a sua seleção no modal; depois um `$dumpvars` escrito à mão no *testbench*, que é respeitado sem qualquer injeção; e, por fim, o padrão, todo o escopo do *testbench*. O seu *testbench* nunca é modificado, pois a instrumentação acontece numa cópia temporária.

14.5 O seletor de `.gtkw`

Ao lado do botão de ondas, o seletor de `.gtkw` escolhe o *layout* ativo: a opção padrão usa o *layout* curado gerado pela AURORA, e Adicionar arquivo .gtkw registra um *layout* seu, salvo de dentro do GTKWave. O *layout* ativo também dirige a precedência do despejo, como visto acima.

14.6 Rodando

Clique em Analisar Verilog e acompanhe no terminal TWAVE a construção, a execução e a abertura do visualizador escolhido; o Capítulo 15 assume daí. Para execuções sem ondas, o Fast Sim, com o Verilator, roda o programa na velocidade máxima e deixa as saídas nos `output_<i>.txt`.

14.7 Teste de hardware

O botão Teste do processador sintetizado valida o processador como caixa-preta: um *harness* C++ do Verilator alimenta as portas de entrada e confere as saídas, sem despejo de ondas, com barra de progresso no THTEST. Para detectar o fim do programa, a AURORA usa o mecanismo do `#TOAQUI` e do pino `cheguei` (Seção 10.2), inserindo o marcador ao final do `main()` quando necessário.

14.8 Simulando o `media_movel`

O roteiro completo do exemplo condutor:

1. crie `media_movel/input_0.txt`, ao lado do fonte, com uma rampa e um degrau, um inteiro por linha;
2. acione Compilar C± e confira nos terminais TCMM e TASM que tudo passou;
3. confirme o motor (Icarus) e o visualizador (GTKWave) nos seletores;
4. acione Analisar Verilog e aguarde o GTKWave abrir com o *layout* curado;
5. observe o histórico `x[0]` a `x[3]`, a variável `soma` e a porta de saída suavizando o degrau, com a trilha C± mostrando a linha do programa executada em cada ciclo.

Os valores de saída também ficam em `media_movel/media_movel/output_0.txt`, na pasta de saída, prontos para conferência num *script*.

PARTE VI

Visualização

CAPÍTULO 15

Formas de onda: GTKWave e Surfer

Depois que a simulação gera o arquivo de ondas, a AURORA abre um visualizador para inspecionar cada sinal do processador ciclo a ciclo. Há dois visualizadores disponíveis. O GTKWave é o padrão e já vem empacotado: um *fork* próprio do laboratório, o `nipscernlab/gtkwave-nipscern`, com tema escuro e ajustes de usabilidade. O Surfer é um visualizador moderno escrito em Rust, opcional, do qual a AURORA usa o *fork* `surfer-aurora`, mantido pelo grupo no GitLab com decodificadores específicos do SAPHO. Ambos leem o mesmo formato de despejo, o FST, de modo que trocar de visualizador não exige simular de novo.

FIGURA 11 *Formas de onda de uma simulação do processador media_movel no GTKWave*

Mostrar as faixas de clock, as trilhas de Assembly e C± em sincronia, as variáveis e as portas.

15.1 O que a AURORA prepara para você

Abrir um despejo bruto num visualizador qualquer significa começar do zero, sem nenhum sinal selecionado e tudo em binário. A AURORA elimina esse trabalho gerando, a cada simulação, um *layout* curado. Cada processador do projeto vira uma seção própria, com divisores nomeados e, no Surfer, um grupo colapsável. Os sinais de topo, *clock*, *reset* e interrupção, vêm primeiro; seguem as portas de entrada e saída em amarelo, as instruções em violeta, as variáveis em laranja e as *flags* ao final. Variáveis do tipo `comp` são decodificadas para a forma legível $a + bi$ com o conversor COMP2GTKW do YANC.

O destaque são as trilhas de texto que mostram, a cada ciclo de *clock*, o mnemônico *assembly* executado e a linha C± correspondente, avançando em sincronia com o circuito: é o seu programa rodando dentro do hardware, visível linha a linha. Essas

trilhas vêm dos arquivos de tradução gerados na compilação. No GTKWave, toda essa curadoria chega como um arquivo `.gtkw`; no Surfer, como um arquivo de estado `.surf.ron`. Nos dois casos, o visualizador já abre pronto para leitura.

15.2 Escolhendo o visualizador

O seletor de visualizador fica na *toolbar*, ao lado do seletor de simulador, e vale para o aplicativo inteiro, lembrado entre sessões. Se o Surfer estiver selecionado mas o executável não estiver presente, a AURORA recua para o GTKWave sem erro.

15.3 GTKWave

O GTKWave empacotado não é o original puro. O *fork* do laboratório traz tema escuro por padrão, com o *canvas* na paleta da AURORA, ajuste automático de *zoom* ao abrir, de modo que a onda inteira caiba na tela, rótulos justificados à esquerda, ícones modernizados e *zoom* animado.

No uso cotidiano, os sinais são adicionados a partir da árvore de escopos à esquerda; o cursor primário se posiciona com o clique e o secundário com o botão do meio, com a barra exibindo a diferença de tempo entre eles; o formato de exibição de cada sinal muda pelo menu de contexto, em Data Format; o *zoom* responde aos botões de lupa e a Ctrl com a roda do mouse; e a seleção atual de sinais, cores e ordem pode ser salva em um `.gtkw` próprio por File, Write Save File. Após uma nova simulação, File, Reload Waveform recarrega o arquivo.

> NOTA
>
> Se você salvar o seu próprio `.gtkw` no projeto e o marcar como ativo no seletor da AURORA, ele passa a ter precedência sobre o *layout* automático: a simulação seguinte grava exatamente os sinais que o seu arquivo referencia (Capítulo 14).

15.4 Surfer

O Surfer é um visualizador com interface acelerada por GPU e carregamento preguiçoso dos arquivos, confortável em simulações grandes. A AURORA baixa o binário do *fork* `surfer-aurora` automaticamente, com verificação de integridade, e o lança com o *layout* curado.

Entre os recursos que valem conhecer estão os formatos de exibição ricos, de binário e hexadecimal a ponto fixo e ponto flutuante IEEE em vários tamanhos; os grupos colapsáveis, um por processador no *layout* da AURORA, essenciais em projetos multiprocessados; as trilhas de *assembly* e C±, instaladas automaticamente como tradutores de mapeamento; os cursores e a navegação rápidos, com uma linha

de comando interna de autocompletar difuso; e o desenho analógico de sinais, útil para ver amostras de sinais processados como curvas. Os complexos do SAPHO também funcionam: a AURORA pré-decodifica os sinais `comp` e instala o resultado como tradutor, de modo que a faixa já abre na forma $a + bi$.

Uma limitação conhecida da versão empacotada no Windows é o recarregamento automático do arquivo de ondas, que não dispara; por isso a AURORA fecha e reabre a janela do Surfer a cada nova simulação. No modal de configuração de ondas há uma opção de manter as janelas abertas, para comparar execuções lado a lado.

CAPÍTULO 16

PRISM: o visualizador de RTL

O PRISM (*Processor Rendering Interface for Schematic Models*) desenha o projeto como um diagrama de circuito navegável, com módulos, portas e conexões, do *top-level* até o interior do processador. É a resposta visual à pergunta sobre que hardware o código virou.

FIGURA 12 *Janela do PRISM com o diagrama do projeto*

O top-level do MeuFiltro renderizado; se possível, um segundo print já dentro do módulo do processador.

16.1 Como abrir

O botão PRISM da *toolbar* exige um *top-level* definido no projeto (Seção 4.4). Acionado, ele compila o que estiver pendente, verifica a sintaxe dos Verilog, reúne todos os fontes do projeto, dos moldes de HDL do SAPHO ao `<proc>.v` na pasta de saída de cada processador, sintetiza com o Yosys em modo diagrama, sem mapeamento tecnológico, para que o desenho preserve a estrutura RTL do código, e renderiza cada módulo como um desenho vetorial antes de abrir a janela. A saída do Yosys corre no terminal TVERI, e a mesma síntese alimenta a visão Hierarquia da árvore de arquivos (Seção 4.5).

16.2 Navegando no diagrama

A janela do PRISM é independente da principal e abre maximizada, no mesmo tema escuro. O mouse faz o deslocamento e o *zoom*. Um clique em qualquer ponto de um módulo, no nome ou no corpo da célula, entra nele, e como os diagramas dos submódulos já foram materializados, a navegação é instantânea; a trilha no topo

mostra o caminho na hierarquia e permite voltar. Um duplo clique numa célula abre o código-fonte correspondente no editor principal, na linha exata da declaração, levando do desenho ao código num gesto. Para ver o que uma célula conecta, Shift mais clique sobre ela, ou a opção Destacar conexões do menu de contexto, acende todos os fios que a tocam; um clique num fio isolado acende só a ligação daquele fio. O botão Recompile refaz a síntese e o diagrama após mudanças. Os blocos do processador usam símbolos próprios, desenhados para a ULA, as memórias e as pilhas, em vez de retângulos genéricos.

16.3 Modo simulação

Além do diagrama estático, o PRISM tem um modo de simulação interativa, apoiado no DigitalJS: o circuito vira um esquemático vivo, no qual se alternam entradas e se observam os valores se propagando pelas conexões. É uma lupa didática para circuitos pequenos, limitada a três mil células; acima disso o PRISM recusa e sugere a visão esquemática comum.

> NOTA
>
> Use o PRISM cedo e com frequência. Compilar o `media_movel` e abrir o diagrama depois de cada mudança torna palpável a relação entre as construções da linguagem e o custo de hardware: acrescente uma divisão ao programa e veja o divisor aparecer no desenho.

PARTE VII

Aurora Intelligence

CAPÍTULO 17

Aurora Intelligence

A Aurora Intelligence é a assistente de IA integrada à AURORA. Ela conversa sobre o projeto, a C±, o Verilog, a arquitetura SAPHO e as mensagens do compilador e, com a sua permissão, age sobre a IDE: edita arquivos, compila, dispara simulações, seleciona sinais de onda e opera o Git. Este capítulo cobre a configuração e o uso.

> INFORMAÇÃO
>
> A assistente funciona no modelo de trazer a própria chave (BYOK, *Bring Your Own Key*): você conecta a sua conta de um provedor de IA. O provedor e os modelos próprios do NIPS-CERN estão no roteiro do laboratório, mas ainda não foram treinados; até lá, a assistente opera sobre provedores externos.

17.1 Abrindo o painel

O botão Assistente IA, a estrela da *toolbar*, abre o painel à direita, empurrando o editor; a largura é ajustável e lembrada entre sessões. O cabeçalho traz o ícone do provedor ativo e os botões de histórico de conversas, nova conversa e fechamento.

FIGURA 13 *Painel da Aurora Intelligence com uma conversa*

Uma pergunta sobre o media_movel respondida, mostrando um cartão de ferramenta executada e o composer embaixo.

17.2 Configurando um provedor

Nas Configurações, a aba Assistente IA exibe um cartão por provedor: OpenAI, Anthropic (Claude), Google (Gemini), DeepSeek, Groq e Ollama, este último local e sem chave, apontando para a instância na sua máquina, com um botão Detect que a encontra junto com os modelos disponíveis. Cada cartão tem o campo da chave,

o campo de modelo, para escolher um específico ou manter o padrão do provedor, e os botões Salvar, Testar, que responde com o tempo de conexão, Obter uma chave, que abre o console do provedor, e Remover chave, além do selo indicando se há chave configurada.

> NOTA
>
> As chaves são cifradas pelo cofre de credenciais do Windows e usadas somente pelo processo principal da AURORA; nunca circulam pela interface nem aparecem em registros. A IDE só exibe se há chave, jamais o valor.

Há uma alternativa às chaves de API: quem assina o Claude Code, da Anthropic, ou o Codex, da OpenAI, pode usar as próprias ferramentas de linha de comando com o login da assinatura. A AURORA baixa a ferramenta sob demanda, com verificação de integridade; a autenticação é o OAuth da própria ferramenta, sem que a IDE veja o token; e as respostas chegam no mesmo painel de conversa, com as ferramentas da AURORA expostas por um servidor local e privado de MCP (*Model Context Protocol*).

17.3 O composer

Na caixa de mensagem ficam os seletores de provedor e de modelo, o controle de esforço de raciocínio para os modelos que o suportam, o indicador de uso da conversa, o seletor de permissões, o botão de anexar arquivos ou imagens e o de interromper a geração; Enter envia. Selecionar código no editor e clicar na estrela que surge junto à seleção envia o trecho já contextualizado (Capítulo 11).

17.4 Ferramentas e permissões

A assistente age por ferramentas curadas, as mesmas operações dos botões da IDE e nada além. Elas se dividem em dois níveis. As de leitura executam sem confirmação: ler o arquivo ativo e os abertos, a árvore do projeto, a saída dos terminais, a lista de processadores, os sinais de onda e as configurações, além de consultar a referência embutida de diretivas, *opcodes* e mensagens do compilador e o estado do Git. As de escrita pedem confirmação: editar, criar, renomear e excluir arquivos, compilar e cancelar compilações, criar processadores e projetos, mudar configurações, selecionar sinais de onda, abrir o Surfer, executar comandos no TCMD e as mutações de Git.

Cada ação de escrita aparece como um cartão de confirmação no chat, com as opções de permitir ou negar, antes de executar. O seletor de permissões do *composer* ajusta o rigor entre confirmar tudo, confirmar apenas escritas, que é o padrão, e permitir tudo.

> CUIDADO
> O modo de permitir tudo deixa a assistente editar arquivos e executar comandos sem perguntar. Use-o apenas em projetos versionados no Git, nos quais qualquer alteração é revisável e reversível (Capítulo 18).

17.5 Conversas

Cada conversa é salva localmente, e o histórico reabre conversas antigas com o contexto completo. Sem conexão, o painel indica que a assistente está *offline*.

17.6 Para que ela é boa no fluxo SAPHO

Alguns usos que rendem bem: perguntar por que um C± não compila, já que ela lê o terminal, conhece as mensagens do YANC e propõe a correção; estimar quanto hardware custa uma função, consultando a referência de *opcodes* e o *assembly* gerado; pedir um *testbench* cocotb que compare o processador com um modelo NumPy; ou simplesmente pedir que selecione os sinais das portas e abra as formas de onda.

PARTE VIII

Ferramentas de apoio

CAPÍTULO 18

Dagr: controle de versão

O Dagr (do nórdico antigo, "dia"; o painel usa a runa *dagaz* como marca) é o painel de controle de versão da AURORA: uma interface completa de Git sobre o repositório do projeto aberto, com integração ao GitHub. Não é preciso instalar nem conhecer a linha de comando do Git para versionar projetos SAPHO, embora o TCMD esteja ali para quem preferir.

18.1 Abrindo

O botão Controle de Versão da *toolbar*, com o ícone de ramificação e um contador de alterações pendentes, abre o modal do Dagr; o indicador do GitHub na barra de status leva ao mesmo lugar. Se o projeto ainda não é um repositório, o painel oferece inicializá-lo, e criar um `.gitignore` adequado está a um clique no menu de contexto da árvore.

FIGURA 14 *Modal do Dagr com alterações pendentes*

A aba Changes com arquivos preparados e não preparados, o diff de um deles e a caixa de commit.

18.2 A aba Changes

O fluxo cotidiano tem três passos. A lista de alterações mostra cada arquivo modificado, criado ou excluído, e o clique abre o *diff* colorido embutido. Em seguida, prepara-se o que vai entrar no *commit*, por arquivo ou de uma vez, com a operação inversa igualmente disponível. Por fim, escreve-se o resumo do *commit*, com descrição opcional, e confirma-se.

Alguns apoios completam a aba: emendar o último *commit*, desfazê-lo mantendo as alterações, descartar as mudanças de um arquivo e avisos para binários e *diffs* muito grandes. A árvore de arquivos participa do sistema, decorando cada arquivo com o seu estado no Git, na cor correspondente.

18.3 A aba History

O histórico de *commits* do ramo atual, com autor, data e mensagem; o clique mostra os arquivos e o *diff* de cada um.

18.4 Ramos e stashes

O painel cria, troca e mescla ramos, e na troca com alterações pendentes oferece guardá-las automaticamente antes de mudar. As alterações guardadas ficam na prateleira de *stashes*, de onde podem ser restauradas ou descartadas.

18.5 GitHub

O botão de entrar com o GitHub aceita dois métodos: o fluxo de dispositivo, no qual a AURORA abre o navegador e mostra o código a digitar, ou um token clássico com escopo de repositório, com um guia embutido de como criá-lo. O token é cifrado no cofre do sistema, e desconectar o remove. Autenticado, o Dagr publica o projeto local como repositório novo no GitHub, sincroniza com *push*, *pull* e *fetch*, com progresso ao vivo e *autostash* no *pull*, e clona repositórios remotos; se o clone contiver um `.spf`, o painel oferece abrir o projeto no SAPHO em seguida.

> NOTA
>
> Versione desde o primeiro dia. O `.spf` é JSON amigável a *diffs*, os fontes são texto, e um repositório é o pré-requisito para usar a Aurora Intelligence com tranquilidade, pois todo passo dela vira um *diff* revisável no Dagr.

A Aurora Intelligence expõe o mesmo motor como ferramentas de Git, com as mutações sujeitas a confirmação (Capítulo 17).

CAPÍTULO 19

Configurações

O botão Configurações do Aurora, na *toolbar*, abre o modal de configurações, organizado em sete seções na navegação lateral, com os botões Restaurar Padrões e Salvar Alterações no rodapé.

FIGURA 15 *Modal de Configurações*

A seção Geral aberta, mostrando a navegação lateral com as sete seções.

A seção Geral traz dois interruptores: o de *tooltips*, ligado por padrão, que controla as dicas exibidas ao passar o mouse pela IDE, e o de confiança em links externos, desligado por padrão, que quando ativado abre os links do chat da assistente no navegador sem pedir confirmação.

A seção Aparência permite trocar o ícone da aplicação por uma imagem sua, ou restaurar o padrão.

> INFORMAÇÃO
>
> A AURORA tem um único tema visual, o tema escuro aurora, e ele não é configurável. Trata-se de uma decisão de projeto da plataforma.

A seção Idioma alterna entre português, o padrão, e inglês. A escolha muda a interface e também as mensagens do compilador YANC, que recebe a opção de idioma correspondente: os erros de compilação chegam na língua da IDE.

A seção Terminal controla o Modo Verboso, ligado por padrão, que exibe todas as mensagens de compilação; desligado, entram em ação os filtros por classe dos terminais (Capítulo 13).

A seção Atalhos de Teclado lista todos os atalhos da IDE e permite regravá-los: clique num atalho, tecle a nova combinação, e combinações em conflito são recusadas com aviso. A lista padrão está no Apêndice C.

A seção Assistente IA reúne os cartões de provedores da Aurora Intelligence, com chaves, modelos e testes de conexão (Seção 17.2).

A seção Sobre, por fim, é a ficha da plataforma: a versão instalada, a equipe de desenvolvimento e orientação, os vínculos institucionais com a UFJF e o NIPS-CERN, os agradecimentos às agências de fomento, os links oficiais e o catálogo completo de software de terceiros com as respectivas licenças, do Icarus Verilog ao Monaco.

PARTE IX

Manutenção da plataforma

CAPÍTULO 20

Atualizações automáticas

O SAPHO evolui rápido, e a AURORA cuida de se manter atual: um atualizador embutido verifica, baixa e instala novas versões, sempre com o seu aval.

20.1 Como funciona, do seu ponto de vista

Cerca de seis segundos após abrir a IDE, a AURORA consulta silenciosamente o canal oficial de *releases*, o repositório `nipscernlab/sapho` no GitHub; se você já está na última versão, nada acontece, nenhum diálogo. Havendo versão nova, abre-se a janela de atualização, com a versão nova e a atual, o tamanho do download e o registro de mudanças bilíngue. Nada é baixado sem você pedir. Ao aceitar, a barra de progresso mostra percentual, velocidade e tempo restante, e é possível continuar trabalhando. Com o download pronto, o botão vira Reiniciar e instalar: a AURORA fecha, o instalador roda e a IDE reabre na versão nova, confirmando a mudança com uma notificação discreta na primeira abertura.

A integridade do pacote é verificada por *hash* criptográfico antes de instalar, e os downloads são incrementais quando possível. Em caso de problema, como falta de internet, de espaço ou de permissão, a janela explica em linguagem clara e a IDE continua funcionando na versão atual.

FIGURA 16 *Janela de atualização*

Com uma versão nova disponível: changelog, tamanho do download e o botão de baixar; se possível, um segundo print com o download em progresso.

> INFORMAÇÃO
>
> Há dois repositórios por desenho: o desenvolvimento acontece em `nipscernlab/aurora`, e as versões estáveis são publicadas em `nipscernlab/sapho`, o canal que o instalador e o atualizador consomem. Você sempre recebe *releases* estáveis, nunca o estado intermediário do desenvolvimento.

20.2 A toolchain acompanha

Cada instalador carrega, embutida, a versão da cadeia de ferramentas testada com aquela versão da IDE: os compiladores do YANC, os simuladores, o GTKWave e os moldes de HDL. Atualizar a AURORA atualiza o conjunto inteiro, em versões validadas juntas, sem nada para gerenciar manualmente.

20.3 Este manual também se atualiza

O manual que você lê é parte do ecossistema versionado: cada mudança relevante na AURORA, no YANC ou nos componentes reflete numa edição nova, e a versão do manual acompanha a da plataforma.

CAPÍTULO 21

Solução de problemas

Um capítulo de pronto-socorro, com os tropeços mais comuns e o caminho de volta. As tabelas seguem o formato sintoma e solução; para os diagnósticos do compilador em si, veja a Seção 10.7.

21.1 Compilação e simulação

Sintoma	Causa e solução
Erro de sintaxe no TCMM	Clique na linha para ir ao ponto. Confira as ausências da linguagem: não há decremento, operadores compostos nem ponteiros (Capítulo 8).
Falta o `main()`	Todo programa C± precisa de `void main()`.
Porta de E/S inexistente	O índice em `in()` ou `out()` excede as diretivas de porta. Aumente `#NUIOIN` ou `#NUIOOU` e recompile.
`#PRNAME` não existe mais	Desde o YANC 7.1 o nome do processador é o nome do arquivo. Apague a linha; para outro nome, renomeie o processador.
O nome do arquivo não serve de processador	O nome vira módulo Verilog: só letras sem acento, dígitos e sublinhado, sem dígito nem `__` no começo, e nenhuma palavra reservada do Verilog. Renomeie o processador.
O fonte não tem o nome do processador	Um `outro.cmm` dentro da pasta do processador `filtro` geraria o módulo `outro`. Renomeie o arquivo para `filtro.cmm`, ou mova-o para um processador com o nome dele.
Arquivo na pasta de saída, mas o terminal mostra erro	O arquivo é de uma compilação anterior. Vale o terminal.

Sintoma	Causa e solução
Total de bits recusado no Hub	Vale a igualdade estrutural (mantissa mais expoente mais um), e o ganho deve ser potência de dois.
O botão de ondas não age	Falta *testbench*, ou *top-level* no caso do PRISM. Compile o C± para gerar o `_tb.v`, ou marque o seu pelo menu de contexto.
A simulação termina cedo demais	Poucos ciclos. Aumente o número de ciclos no painel de configurações do processador.
Simulação lenta demais	Troque o motor para o Verilator, ou reduza os sinais gravados na configuração de ondas.
Sinais internos sumiram das ondas	Você está no Verilator, que expõe apenas o topo do *testbench*. Volte ao Icarus para depurar por dentro.
Processo travado	O botão Cancelar interrompe toda a cadeia sem fechar a IDE.
Binário do simulador não encontrado	A pasta `components` da instalação está incompleta ou foi movida. Reinstale o SAPHO pelo instalador oficial.

21.2 Projeto e arquivos

Sintoma	Causa e solução
Projeto antigo pede conversão ao abrir	O projeto ainda tem `Software`, `Hardware` e `Simulation`, ou usa `#PRNAME`. Aceite a conversão e leia o relatório (Capítulo 4); sem ela, o projeto não compila.
Projeto não encontrado	O `.spf` foi movido ou apagado. Abra pelo novo caminho; o item sai dos recentes sozinho.
Aviso de arquivos faltantes	Arquivos registrados no `.spf` sumiram do disco. Restaure-os, ou dispense o aviso para limpar as referências.

Sintoma	Causa e solução
Arquivo modificado externamente	Outro programa alterou um arquivo aberto. Escolha entre manter a versão do editor, usar a do disco, ou salvar e recarregar.
Arrastar para a árvore não funciona	A importação por arraste aceita `.v`, `.sv`, `.vh` e `.py`. Arquivos `.gtkw` entram pelo seletor da *toolbar*.

21.3 Visualizadores

Se você selecionou o Surfer e o GTKWave abriu, o executável do Surfer não está presente, e a AURORA recuou de propósito; reinstale ou atualize para obter o componente. Se o Surfer não recarregou a onda nova, trata-se do comportamento esperado no Windows, no qual a AURORA fecha e reabre a janela a cada simulação. Se o PRISM recusou o modo de simulação, o circuito passa de três mil células; use a visão esquemática comum.

21.4 Aurora Intelligence

Quando o painel indica que a assistente está *offline*, não há conexão com o provedor; verifique a rede e use o botão de teste da chave nas Configurações. Quando um modelo está indisponível, a AURORA recua automaticamente para o padrão do provedor, e outro pode ser escolhido no seletor. Quando o Ollama não é detectado, confirme que o serviço local está em execução e use o botão Detect no cartão correspondente.

21.5 Onde estão os registros

Para relatar um problema, anexe o registro principal, em `\%APPDATA\%\textbackslashSAPHO\textbackslashlogs\textbackslashmain.log`; o anterior fica em `main.old.log`. Mais simples que digitar o caminho: em Configurações > Sobre > Atualizações, o botão Abrir log revela o arquivo direto no explorador. O conteúdo dos terminais pode ser salvo pelo botão de exportação. Relatos são bem-vindos nos repositórios da organização `nipscernlab`, no GitHub.

PARTE X

Apêndices

CAPÍTULO A

Referência de diretivas

A.1 Diretivas de configuração

Uma por linha, no topo do `.cmm`, sem ponto e vírgula. A coluna do padrão indica o valor assumido pelo compilador quando a diretiva é omitida. Faltar uma diretiva não dá erro: o processador nasce com o padrão, que pode não ser o que você quer. Por isso o Hub de Processadores da AURORA escreve sempre `#NUBITS`, `#NBMANT`, `#NBEXPO`, `#NUIOIN` e `#NUIOOU`, e o `#NUGAIN` só quando difere de 128; as pilhas ficam com o valor calculado.

Diretiva	Argumento	Padrão	Efeito
`#NUBITS`	inteiro	32	Largura da palavra da ULA e dos inteiros, em complemento de dois; deve valer a soma de mantissa, expoente e um.
`#NBMANT`	inteiro	23	Bits de mantissa do ponto flutuante próprio.
`#NBEXPO`	inteiro	8	Bits de expoente, em complemento de dois.
`#NDSTAC`	inteiro	calculado	Profundidade da pilha de dados; sem a diretiva, o compilador usa o pico do programa mais um, no mínimo dois.
`#SDEPTH`	inteiro	calculado	Profundidade da pilha de sub-rotinas, calculada do mesmo jeito.
`#NUIOIN`	inteiro	1	Número de portas de entrada; `in(p)` exige `p` menor que esse valor.
`#NUIOOU`	inteiro	1	Número de portas de saída; idem para `out`.

Diretiva	Argumento	Padrão	Efeito
`#NUGAIN`	potência de 2	128	Divisor fixo da função `norm()`; outro valor que não seja potência de dois é recusado.
`#FFTSIZ`	inteiro	3	Bits invertidos no endereçamento bit-reverso `x[k]`, para FFT de 2^n pontos.

Não existe `#PRNAME`: o nome do processador é o nome do arquivo fonte (`filtro.cmm` gera o processador e o módulo `filtro`), e a diretiva, se aparecer, é erro de compilação. O nome aceita letras sem acento, dígitos e sublinhado, não pode começar com dígito nem com `__` e não pode ser uma palavra reservada do Verilog ou do SystemVerilog (`wire`, `module`, `begin`...).

Com uma única porta a ligação é direta; com mais de uma, o hardware instancia automaticamente um decodificador de endereços.

A.2 Diretivas comportamentais

Aparecem no corpo do programa, marcando pontos do código. A `#PRACA` marca o ponto para onde o processador desvia quando o pino de interrupção pulsa. A `#TOAQUI` marca um endereço do programa cujo alcance faz pulsar o pino `cheguei`; é também o mecanismo que a AURORA usa para detectar o fim do programa em testes e simulações.

A.3 Pré-processador

A construção `#define NOME corpo` define uma constante simbólica, apenas na forma de objeto, sem argumentos, com o limite de 256 por programa. Não há `#include` nem `#ifdef` no fluxo C±.

CAPÍTULO B

Referência da biblioteca padrão

Todas as funções intrínsecas da C±, agrupadas. Erros de tipo são apontados na compilação; as restrições de cada grupo estão no Capítulo 9.

B.1 Entrada e saída

Função	Tipos	Descrição
`in(p)`	`int`	Lê a porta de entrada `p`.
`fin(p)`	`float`	Lê a porta `p` convertendo para `float`.
`out(p, x);`	qualquer	Escreve `x` na porta de saída `p`.
`fout(p, x);`	`float`	Escreve em formato `float`.

B.2 Links entre processadores

Função	Tipos	Descrição
`toma(k, x);`	qualquer	Entrega `x` pelo *link* `k` e espera o parceiro pegar.
`cade(k)`	`int`	Espera o *link* `k` oferecer um valor e o recebe.
`fcade(k)`	`float`	O mesmo, recebendo como `float`.
`toma_sync(k, x);`	qualquer	As formas `_sync` das três: para parceiro no mesmo *clock*, sem sincronizador, 4 ciclos a menos por valor.
`cade_sync(k)`	`int`	

Função	Tipos	Descrição
`fcade_sync(k)`	`float`	

O número do *link* é um literal inteiro; detalhes e ligação no *top-level* na Seção 10.5.

B.3 Funções especiais

Função	Tipos	Descrição
`norm(x)`	`int`	Divide pelo valor de `#NUGAIN` sem instanciar o divisor.
`pset(x)`	`int`, `float`	Devolve x se não negativo; senão zero.
`abs(x)`	todos	Valor absoluto; magnitude para `comp`.
`sign(x, y)`	`int`, `float`	Devolve y com o sinal de x.
`copy(x, y);`	todos	Cópia bit a bit, sem checagem de tipo.

B.4 Não lineares e arredondamento

Função	Descrição
`sqrt(x)`	Raiz quadrada, pelo método de Newton; não aceita `comp`.
`sin(x)`, `cos(x)`, `tan(x)`	Trigonométricas.
`atan(x)`	Arco-tangente.
`sinh(x)`, `cosh(x)`, `tanh(x)`	Hiperbólicas.
`exp(x)`, `log(x)`	Exponencial e logaritmo natural; não aceitam `comp`.
`pow(x, y)`	Potência; a estratégia depende do expoente (Capítulo 9).
`floor(x)`, `ceil(x)`, `round(x)`	Arredondamentos; retornam `float`, e o `round` afasta o meio do zero.

B.5 Complexos

Função	Descrição
`real(z)`, `imag(z)`	Partes real e imaginária.
`fase(z)`	Argumento (ângulo).
`mod2(z)`	Magnitude ao quadrado.
`complex(re, im)`	Constrói um `comp`.
`conj(z)`	Conjugado.

B.6 Notação de Dirac

Sintaxe	Operação
`<a|b>`	Produto interno (expressão).
`a # |0>;`	Zera o vetor.
`a # |M|b>;`	Matriz vezes vetor.
`a # c|b>;`	Vetor escalado.
`A # |a><b|;`	Produto externo.
`out(p, c|a>);`	Emite o vetor escalado pela porta `p`.

B.7 Famílias de instruções do processador

Para a leitura das trilhas de *assembly* nas ondas, os mnemônicos seguem convenções de prefixo e sufixo: `F_` indica a versão em ponto flutuante; `S_` e `SF_`, o operando vindo da pilha; `P_`, o empilhamento prévio; `_M`, a operação sobre memória; e `_V`, a variante indexada.

Família	Mnemônicos principais
Carga e armazenamento	`LOD`, `SET`, `LDI`, `STI`, `ILI`, `ISI`
Pilha	`PSH`, `POP`
Entrada e saída	`INN`, `F_INN`, `OUT`
Links	`TOM`, `CAD` (o operando é o número do *link*)

Família	Mnemônicos principais
Fluxo	`JMP`, `JIZ`, `CAL`, `RET`, `NOP`
Aritmética inteira	`ADD`, `MLT`, `DIV`, `MOD`, `NEG`, `ABS`, `SGN`, `PST`, `NRM`
Aritmética em ponto flutuante	`F_ADD`, `F_SU1`, `F_SU2`, `F_MLT`, `F_DIV`, `F_NEG`, `F_ABS`, `F_ROT`, `F_SCL`, `XPO`
Conversões	`I2F`, `F2I`
Lógica e bits	`AND`, `ORR`, `XOR`, `INV`, `LAN`, `LOR`, `LIN`
Comparações	`LES`, `GRE`, `EQU`, e as versões `F_`
Deslocamentos	`SHL`, `SHR`, `SRS` (o operador `»`)

Cada mnemônico usado pela primeira vez liga o bloco de hardware correspondente no processador gerado; a lista completa e o percentual de uso da ISA aparecem no terminal TASM a cada compilação.

CAPÍTULO C

Atalhos de teclado

Padrões de fábrica, todos reconfiguráveis nas Configurações, na seção Atalhos de Teclado.

Atalho	Ação
Ctrl+N	Novo arquivo
Ctrl+S	Salvar
Ctrl+Shift+S	Salvar todos
Ctrl+W	Fechar aba
Ctrl+Shift+T	Reabrir última aba fechada
Ctrl+Shift+F	Buscar nos arquivos do projeto
Ctrl+Shift+P	Paleta de comandos
Ctrl+Alt+S	Ligar ou desligar a análise semântica (slang)
Shift+Alt+F	Formatar documento (clang-format)

Dentro do editor valem ainda os atalhos usuais do Monaco, herdados do VS Code: Ctrl+F para localizar, Ctrl+H para substituir, Ctrl+Z e Ctrl+Y para desfazer e refazer, Alt+Click para múltiplos cursores e Ctrl+/ para comentar a linha, entre outros.

CAPÍTULO D

Glossário

SAPHO
: *Scalable-Architecture Processor for Hardware Optimization*: o processador *soft-core* e, por extensão, a plataforma completa e o canal de distribuição.

AURORA
: *Advanced Utility Running Optimized Resource Architectures*: a IDE de desktop da plataforma.

YANC
: *Yet Another Compiler*: a suíte de compiladores que traduz C± e C++ em processador Verilog.

C± (CMM)
: A linguagem de programação da plataforma, dialeto de C com complexos nativos e notação de Dirac; arquivos `.cmm`.

PRISM
: *Processor Rendering Interface for Schematic Models*: o visualizador de RTL da AURORA.

Aurora Intelligence
: A assistente de IA integrada à AURORA.

Dagr
: O painel de controle de versão (Git) da AURORA.

NIPS-CERN
: O Núcleo de Instrumentação e Processamento de Sinais da UFJF, em parceria com o CERN; o laboratório que desenvolve a plataforma.

soft-processor
: Processador descrito em HDL e implementado na malha reconfigurável de um FPGA; também dito *soft-core*.

FPGA
: *Field-Programmable Gate Array*: circuito integrado reconfigurável.

Verilog
: A linguagem de descrição de hardware (HDL) usada pela plataforma.

.SPF *SAPHO Project File*: o arquivo JSON que define um projeto.

.ASM
O *assembly* intermediário do SAPHO, entre o compilador e o montador.

.MIF *Memory Initialization File*: imagem de memória, de programa ou de dados, embutida no circuito.

TESTBENCH
O banco de testes da simulação, em Verilog (`_tb.v`) ou em Python (cocotb), que estimula o processador.

TOP-LEVEL
O módulo Verilog que integra os processadores e blocos do projeto; o topo da síntese.

ULA
Unidade lógica e aritmética; no SAPHO, apenas os operadores usados pelo programa são instanciados.

ACC
O acumulador, registrador central do processador, destino de toda operação da ULA.

HARVARD
Arquitetura com memórias de programa e de dados separadas.

ISA *Instruction Set Architecture*: o conjunto de instruções do processador.

ICARUS VERILOG
O simulador de Verilog padrão da plataforma; compila com o `iverilog` e executa com o `vvp`.

VERILATOR
Simulador de alto desempenho que converte o circuito em executável nativo.

COCOTB
Framework Python para a escrita de *testbenches*.

YOSYS
Ferramenta de síntese usada pelo PRISM e pela visão Hierarquia.

GTKWAVE
Visualizador de formas de onda padrão, em *fork* próprio do laboratório.

SURFER
Visualizador de formas de onda opcional, em Rust, no *fork* `surfer-aurora`.

VCD, FST
Formatos de arquivo de ondas gerados pela simulação, textual e binário compacto.

.GTKW, .SURF.RON
Arquivos de *layout* de ondas do GTKWave e do Surfer: quais sinais, cores e

grupos mostrar.

Monaco
: O motor de edição de código da AURORA, o mesmo do VS Code.

LSP
: *Language Server Protocol*: o mecanismo dos diagnósticos e da navegação de código (Verible, slang).

BYOK
: *Bring Your Own Key*: o modelo de chaves da Aurora Intelligence.

MCP
: *Model Context Protocol*: como as ferramentas de linha de comando de IA acessam as ferramentas da AURORA.

bit-reverso
: Endereçamento com os bits do índice invertidos, `x[k]`, usado em FFT.

DLP
: Dispositivos Lógicos Programáveis: a disciplina da UFJF que usa a plataforma no ensino.